유승민
(1998.05.23)

CONTACT:
+82 -10-8262-7443
anbak98@gmail.com

기술:https://anbak98.tistory.com/
개인:https://anbak98.naver.com/
취미:https://anbak98.github.io/

2019.03 – 2019.12 스마일게이트 멤버십

2019.03 – 2022.01 게임 제작 소학회 TML 임원 및 C 스터디장

2021.01 – 2022.01 게임 창업 동아리 Coin-On 팀장

2021.07 – 2022.01 머신러닝 게임 레벨 디자인 자기주도연구

2023.12.07 Ajou softcon AI 집중교육 부문 최우수상

2023.12.13 GEEKS 2023 게임부문 대상

2024.08.17 GIGDC 2024 제작부문 동상

2019.03 – 2025.02 아주대학교 소프트웨어융합학과 학사 과정

나에게 게임이란?

: ***그 무엇보다 즐거운 모험이며, 내 삶의 일부이다.***

나에게 게임 개발이란?

: ***게임은 관념이고, 유저는 사유하며, 기술은 수단이다. 유저를 벗어나 게임을 만드는 것은 또 다른 모험이지만 동시에 고행이다.***

주 사용 게임 엔진

주 사용 언어

그 외

#C #Java #WebGL #HTML5 #JS #Flask #Spring #SIC&SIC/XE #Pytorch #Tensorflow #UnityNetcode #MySQL #UnityMLAgent #RPGMaker #bash #Phaser3 #Trello #Aseprite #FLStudio #Slack #Microsoft Teams #Word #Excel #GitHub #GitLab #Clip Studio #eclipse #Anaconda #VS #VSC #GoogleColab #JupyterNotebook #Notion #Obsidian #WireShark

2024

Big-Defense

개요	매우 큰 맵에서 디펜스 게임
참여	개인
역할	All
기간	2024.08~
환경	Unity

비고

https://github.com/Anbak98/Unity-Big-Defence/commits/main/

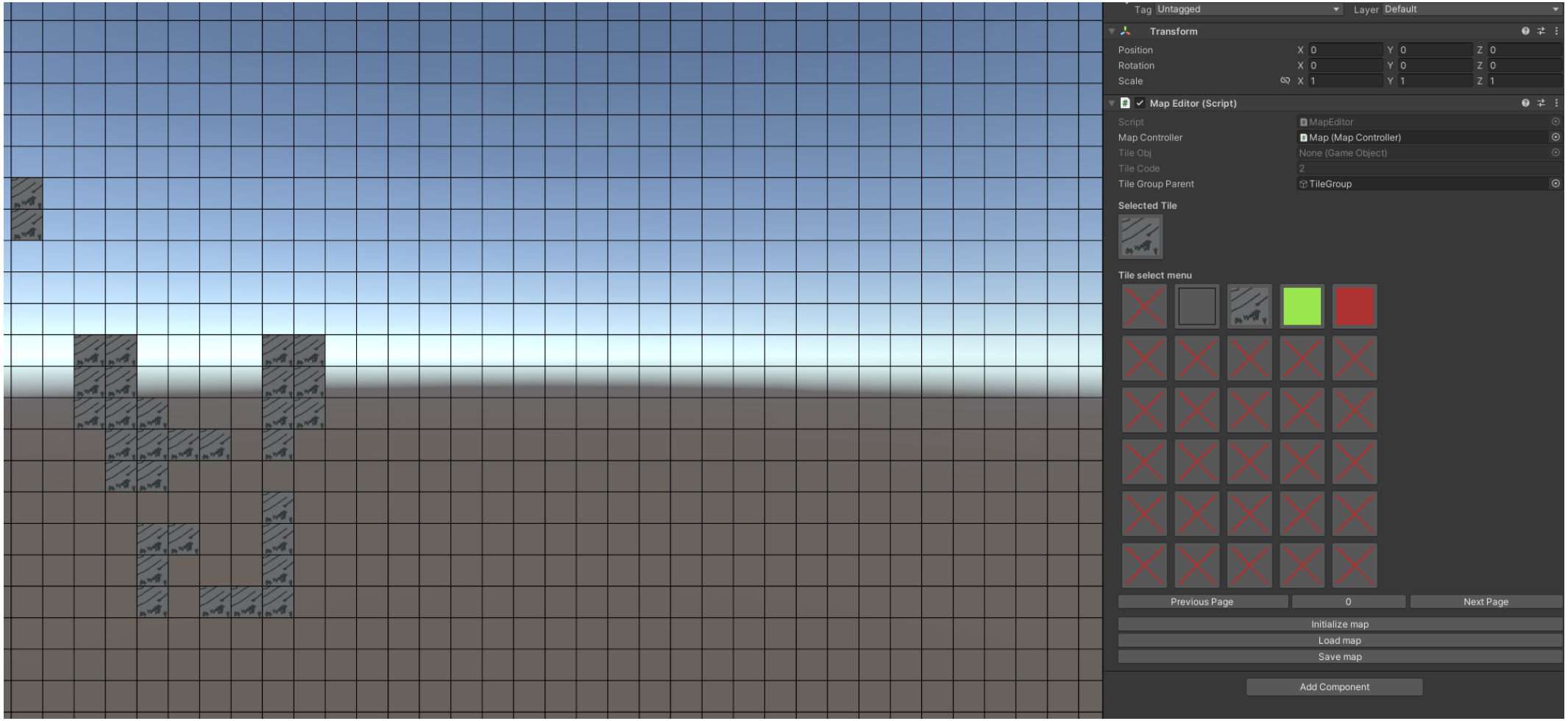
개발 중

리소스가 없어서 대신 내 얼굴을 사용 중

코드

```
[ExecuteInEditMode]
Unity 스크립트(자산 참조 1개)|참조 3개
public class MapEditor : MonoBehaviour
```

먼저 디펜스 Map을 만들 수 있는 MapEditor를 구현했다.



Map은 2차원 배열 데이터를 기반으로 구축되며,
우측 인스펙터는 커스터마이징 한 것으로 OnInspectorGUI()를 오버라이드해 타일 선택 기능을 추가했다.
타일을 선택하고 맵의 셀을 클릭하면 그 셀에 타일이 삽입된다.

코드

Map은 에셋으로 저장할 수 있게 ScriptableObject를 상속받았다. 또한 맵 데이터는 2차원 배열 형태로 Tile 정보, Tower 정보, Grid 정보 세 개로 나뉘어 저장된다.

```
[Serializable]
[CreateAssetMenu(fileName = "MapSaveData", menuName = "Custom/SaveData/Map", order = 1)]
Unity 스크립트|참조 2개
public class MapSaveData : ScriptableObject
{
    [ReadOnly][SerializeField]
    private bool isInitialized = false;
    참조 0개
    public Vector2Int CorePoint { get; private set; }

    참조 1개
    [field: ConditionalShow("isInitialized")][field: SerializeField] public int TopLength { get; private set; }
    참조 1개
    [field: ConditionalShow("isInitialized")][field: SerializeField] public int BottomLength { get; private set; }
    참조 1개
    [field: ConditionalShow("isInitialized")][field: SerializeField] public int LeftLength { get; private set; }
    참조 1개
    [field: ConditionalShow("isInitialized")][field: SerializeField] public int RightLength { get; private set; }

    참조 14개
    public int Width { get; private set; }
    참조 20개
    public int Height { get; private set; }

    [ConditionalShow("isInitialized")][SerializeField] private int[] gridCodeSaveData;
    [ConditionalShow("isInitialized")][SerializeField] private int[] tileCodeSaveData;
    [ConditionalShow("isInitialized")][SerializeField] private int[] buildingCodeSaveData;

    참조 13개
    public int[,] TileCode { get; set; }
    참조 6개
    public int[,] GridCode { get; set; }
    참조 9개
    public int[,] BuildingCode { get; set; }
```

가독성과 안정성을 높이기 위해 한 번 사이즈가 설정된 맵은 사이즈가 다신 바뀌지 않게 할 필요가 있었다 때문에 커스텀 애트리뷰트를 구현했다.

```
참조 13개
public class ConditionalShowAttribute : PropertyAttribute
{
    참조 3개
    public string ConditionFieldName { get; private set; }

    참조 7개
    public ConditionalShowAttribute(string conditionFieldName)
    {
        ConditionFieldName = conditionFieldName;
    }
}

[CustomPropertyDrawer(typeof(ConditionalShowAttribute))]
참조 0개
public class ConditionalShowDrawer : PropertyDrawer
{
    참조 0개
    public override void OnGUI(Rect position, SerializedProperty property, GUIContent label)
    {
        ConditionalShowAttribute condAttribute = (ConditionalShowAttribute)attribute;
        SerializedProperty conditionProperty = property.serializedObject.FindProperty(condAttribute.ConditionFieldName);

        if (conditionProperty != null && !conditionProperty.boolValue)
        {
            GUI.enabled = true;
            EditorGUI.PropertyField(position, property, label);
        }
        else
        {
            GUI.enabled = false; // GUI를 비활성화하여 읽기 전용으로 만듭니다.
            EditorGUI.PropertyField(position, property, label);
            GUI.enabled = true; // GUI를 다시 활성화합니다.
        }
    }

    참조 0개
    public override float GetPropertyHeight(SerializedProperty property, GUIContent label)
    {
        ConditionalShowAttribute condAttribute = (ConditionalShowAttribute)attribute;
        SerializedProperty conditionProperty = property.serializedObject.FindProperty(condAttribute.ConditionFieldName);

        if (conditionProperty != null && !conditionProperty.boolValue)
        {
            return EditorGUI.GetPropertyHeight(property, label, true);
        }
        else
        {
            return EditorGUI.GetPropertyHeight(property, label, true);
        }
    }
}
```

코드

Map의 Save 와 Load 기능은 시리얼라이즈 가능하고 캡슐화된 1차원 배열에 저장된 값을 기반으로 작동한다.
Private한 1차원 배열 대신 public한 2차원 배열을 외부에서 접근해 get하거나 set할 수 있다.

```
참조 1개
public void Load()
{
    if (AnySaveIsNull()) return;
    if (AnyCodeIsNull()) return;

    for (int x = 0; x < Width; x++)
    {
        for (int y = 0; y < Height; y++)
        {
            TileCode[x, y] = tileCodeSaveData[x * Height + y];
            GridCode[x, y] = gridCodeSaveData[x * Height + y];
            BuildingCode[x, y] = buildingCodeSaveData[x * Height + y];
        }
    }
}

참조 1개
public void Save()
{
    if(AnyCodeIsNull()) return;
    if(AnySaveIsNull()) return;

    for (int x = 0; x < Width; x++)
    {
        for (int y = 0; y < Height; y++)
        {
            tileCodeSaveData[x * Height + y] = TileCode[x, y];
            gridCodeSaveData[x * Height + y] = GridCode[x, y];
            buildingCodeSaveData[x * Height + y] = BuildingCode[x, y];
        }
    }
}
```

코드

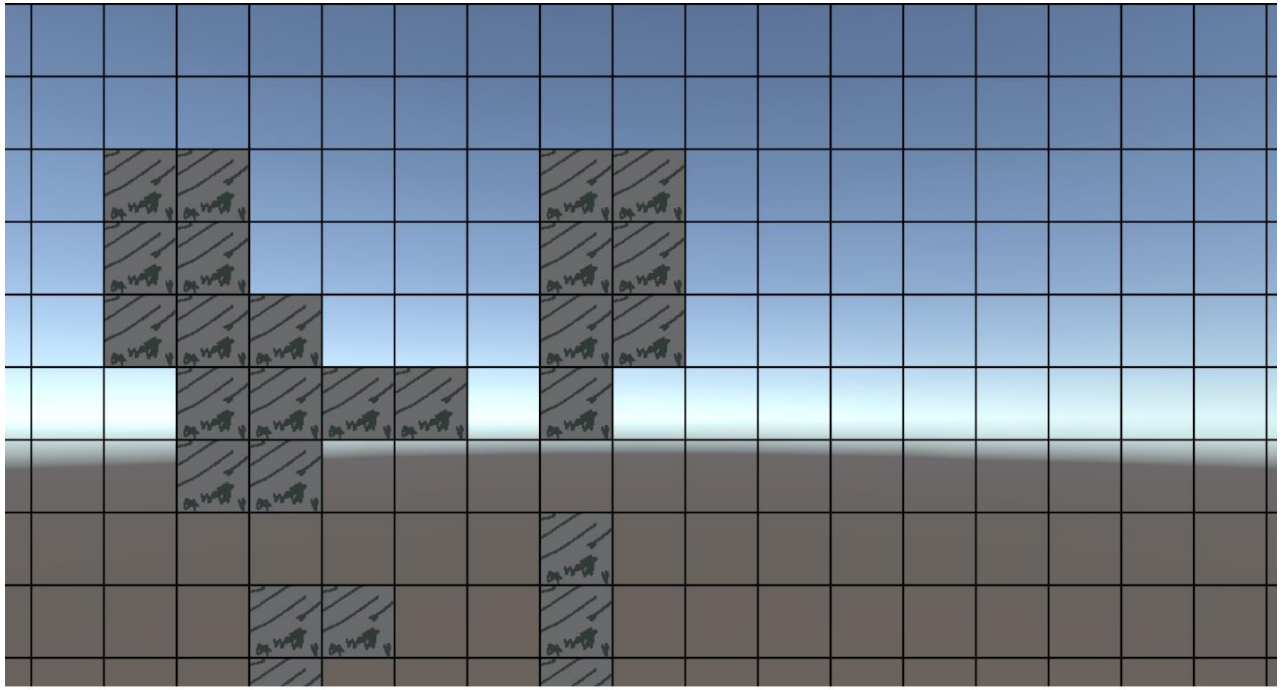
```
if (currentEvent.button == 0 && currentEvent.isMouse) // 좌클릭
{
    switch (currentEvent.type)
    {
        case EventType.MouseDown:

            if (mapController.IsTileInGrid(mousePosTile))
            {
                InstantiateTileObj(tileCode);
                mapController.TileFollowPosition(tileObj, mousePosTile);
                mapController.TileOnPosition(tileObj, tileCode, mousePosTile);
            }
            break;

        case EventType.MouseDrag:
            break;
    }
}
```

MapEditor 씬에서 로드된 맵의 타일을 좌클릭하면 인스펙터에서 선택한 타일이 맵에 추가된다.

Editor 모드에서의 input 감지는 play mode일 때와 달라서 처음에 혼란스러웠다.

심지어 Editor일 때와 Play일 때는 좌표계도 다르게 적용한다. 때문에 좌클릭 했을 때 엉뚱한 부분에 타일이 설치되서 당황했던 기억이 있다.

코드



맵을 만든 후, 플레이 씬으로 넘어와 플레이 버튼을 누르면 맵이 생성된다.

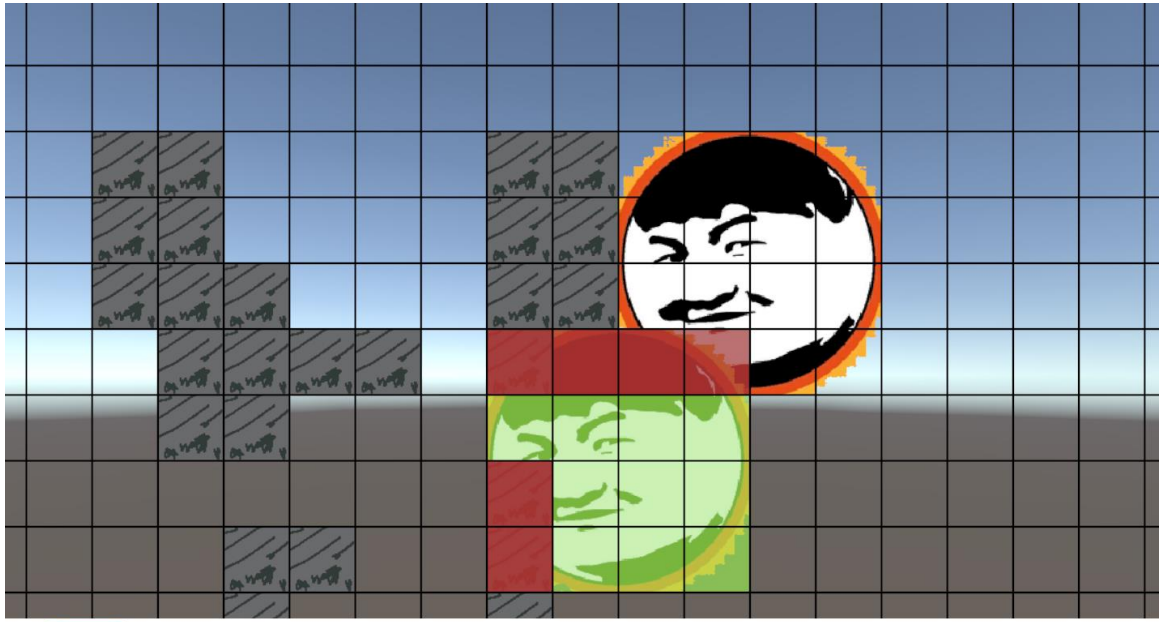
맵을 오브젝트로 생성하는 것은 Generator 타입의 스크립트가 해준다.

```
참조 1개
public GameObject[,] GenerateMapTileGroup(int[,] tileCode, bool? editorMode = false)
{
    int w = tileCode.GetLength(0);
    int h = tileCode.GetLength(1);

    GameObject TileParent = new("TileGroup");
    TileParent.transform.parent = transform;

    if (editorMode.Value)
    {
        TileParent.tag = "EditorOnly";
    }

    GameObject[,] tileObjectGroup = new GameObject[w, h];

    for (int x = 0; x < w; x++)
    {
        for (int y = 0; y < h; y++)
        {
            int code = tileCode[x, y];
            if(code == 0) continue;

            Vector3 position = new(x , y, 0);
            GameObject tileObject = Instantiate(tileData.PrefabList[code], position, Quaternion.identity);
            tileObject.name = $"Tile({tileCode})({x},{y})";
            tileObject.transform.parent = TileParent.transform;

            tileObjectGroup[x, y] = tileObject;
        }
    }

    return tileObjectGroup;
}
```

Map을 Load한 후 tileCode를 get해 이 함수에 넘겨주면 게임 오브젝트를 생성해준다.

이 함수를 호출해주는 것은 Generator 스크립트를 가지고 있는 Controller이다.

코드



```
참조 4개
public void StartBuild(int buildingCode)
{
    if (waitingBuilding)
    {
        Destroy(waitingBuilding);
    }

    if(buildRoutine != null)
    {
        StopCoroutine(buildRoutine);
    }

    detectBuildableTile.UnsetEffectTile();

    waitingBuilding = buildingGenerator.InstantiateBuildingObject(buildingCode, cursorPosition.tilePosOnMouse);

    buildRoutine = StartCoroutine(BuildRoutine(waitingBuilding, buildingCode));
}

참조 1개
public IEnumerator BuildRoutine(GameObject building, int buildingCode)
{
```

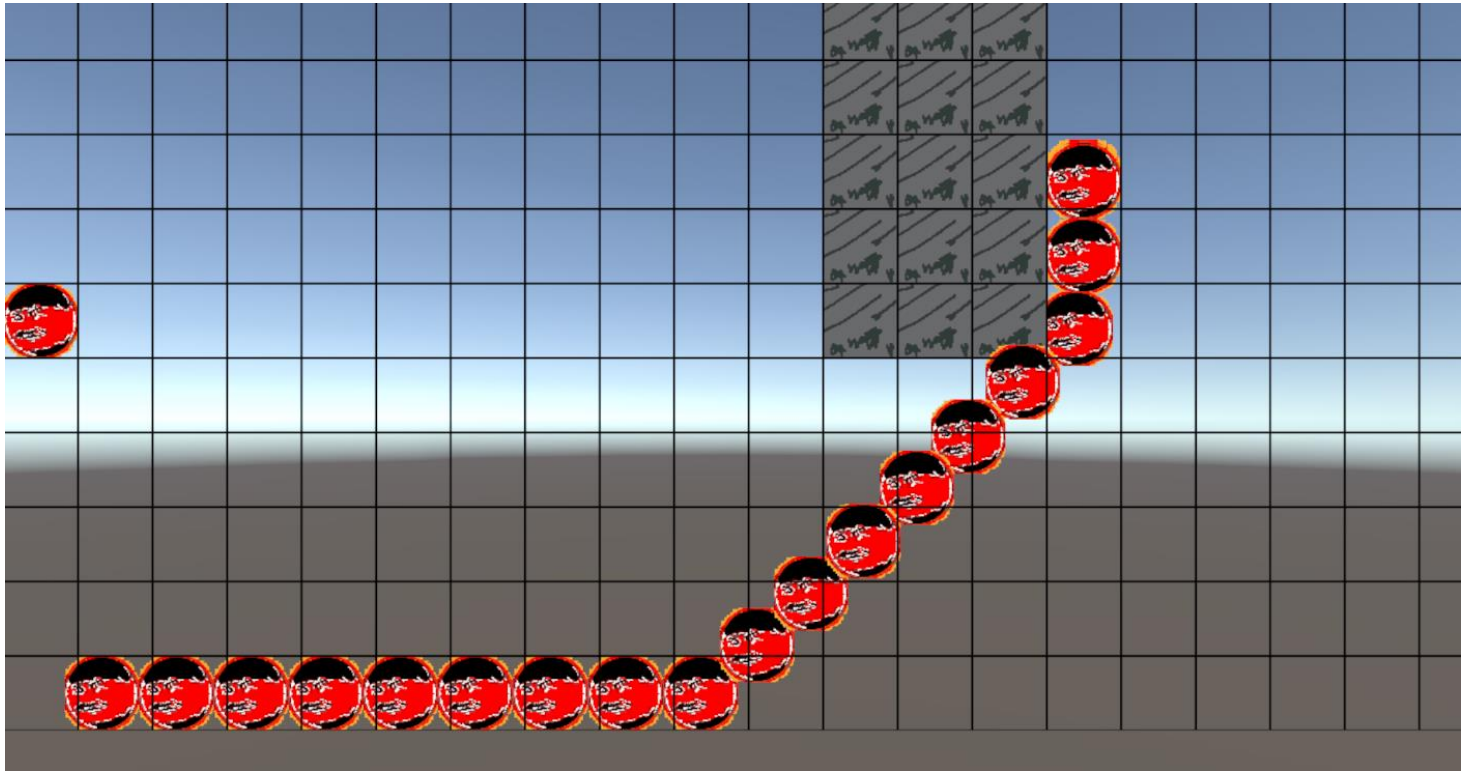
플레이 모드에서는 맵에 타워를 건설할 수 있다. 이 또한 Generator가 해준다.

타워 건설이 불가한 타일은 붉은 색으로 표시해준다.

이를 사용하기 위해선 버튼의 onclick.AddListener() 에 StartBuild() 함수를 추가하기만 하면 된다. Client 측에서 사용하기 쉬운 구조다.

타워를 설치하거나 우클릭을 하면 건설 대기 상태가 취소된다. 즉, 코루틴이 종료된다.

코드



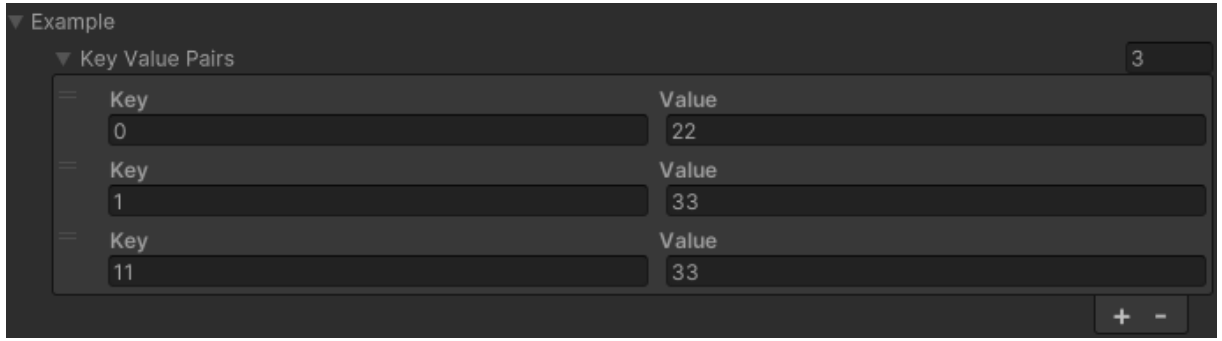
단순 A* 알고리즘을 통해 목적지로 향하는 몬스터들이다. 클라이언트 측에서 프리팹을 가져다가 배치한 후, 목적지 주소와 소환 개수 및 소환 텀만 입력하면 된다.

나는 맵이 큰 만큼 대규모 몬스터를 소환하고 싶다. 따라서 여러 최적화 기법을 공부 중에 있으며, 특히 Compute Shader를 활용한 GPU 연산에 흥미를 두고 있다.

코드

```
using System;
using System.Collections.Generic;
using UnityEngine;

[Serializable]
public class SerializableKeyValue<K,V>  where K : class
{
    public K Key;
    public V Value;
}

[Serializable]
public class SerializableDictionary <K,V> : Dictionary<K,V>, ISerializationCallbackReceiver wh
{
    [SerializeField] private List<SerializableKeyValue<K, V>> _keyValueList;

    public void OnBeforeSerialize()
    {
        if (this.Count < _keyValueList.Count)
        {
            return;
        }

        _keyValueList.Clear();

        foreach (var kv in this)
        {
            _keyValueList.Add(new SerializableKeyValue<K, V>()
            {
                Key = kv.Key,
                Value = kv.Value
            });
        }
    }
```

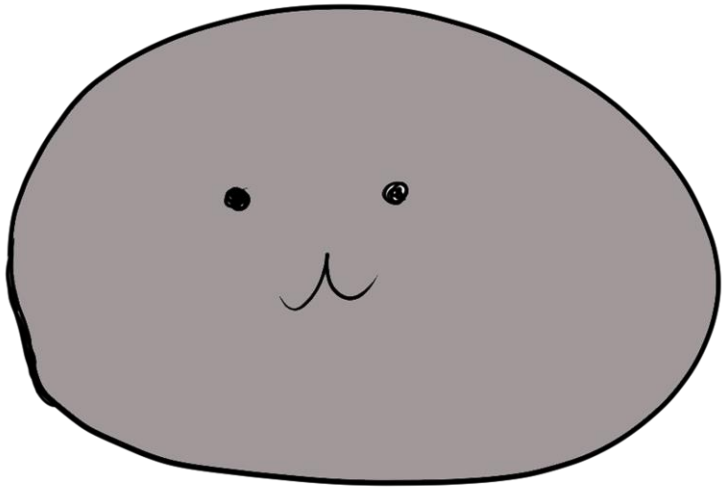
```
    public void OnAfterDeserialize()
    {
        this.Clear();
        foreach (var kv in _keyValueList)
        {
            if(!this.TryAdd(kv.Key, kv.Value))
            {
                Debug.LogError($"List has duplicate Key");
            }
        }
    }
}
```



추가로, Dictionay를 inspector에서 수정하기 위해 ISerializationCallbackReceiver을 상속받아 커스텀 딕셔너리를 만들었다.

2024

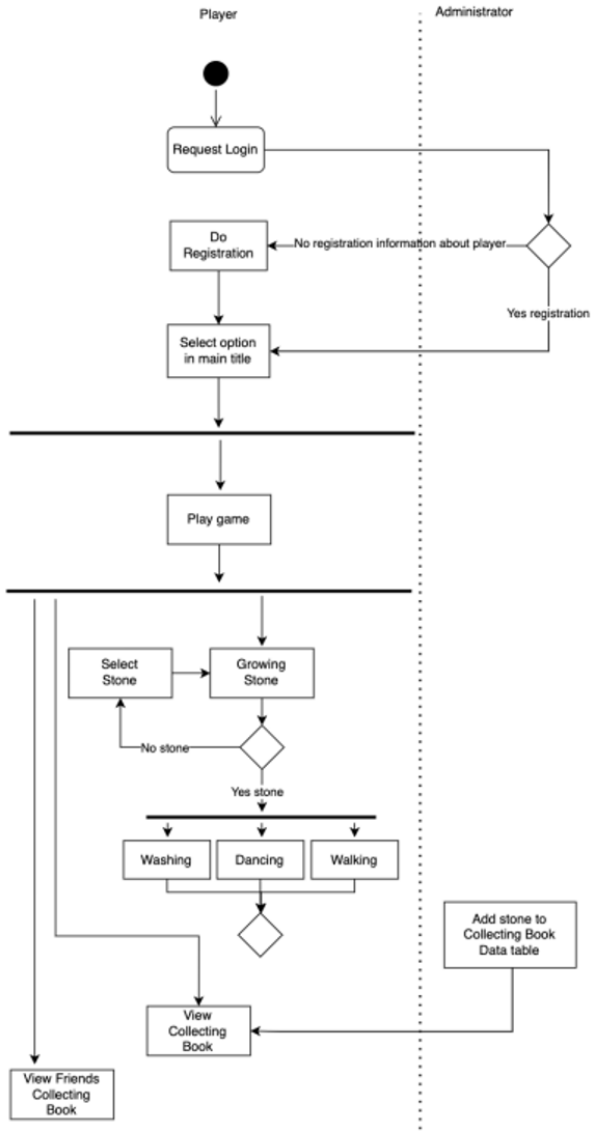
찜질방에서 돌 키우기



개요	찜질방에서 애완 돌을 키우는 게임
참여	2 Client, 2 Server
역할	Client
기간	2024.03.05~2024.06.13
환경	4학년 미디어 소프트웨어 엔지니어링 수업

비고
2024 아주대학교 미디어 소프트웨어 엔지니어링 수업

내 활동



UML Diagram
(Activity & User Case)

Main Programmer. 게임의 전체적인 구조를 설계하고 팀원과 공유함.

00.Manager, 02.Controller, 07.CollectiongBook 폴더에 포함된 스크립트 및 MainTitle과 MiniGame을 제외한 씬들을 작업했다. (별첨)

Git Repository 관리 담당 Unity 측에서 서버 데이터베이스와 통신하도록 함

1. singleton

Our game has three singleton objects. In 'Managers", many manger scripts included with game-manager singleton.

- SoundManager
  - Audio Source
- Managers
- DataManager

2. strategy

Our game has a stone class that adapts strategy patterns. In stone-manager scripts, growing-Stone variable is set by both Lime-Stone class or Granite class. Each stone has methods like increase-HP, check-Evolution.

```
[SerializeField] public Stone growingStone;
```

3. object Pool

Our game used an object pool design pattern when generating obstacles that appear in mini-games. In ObjectPool script, Create and use 5 objects each depending on the type of obstacle.

```
private List<GameObject> CreatePool(GameObject prefab)

//Get an inactive object from the pool, or create a new one if necessary
0 references
public GameObject GetPooledObject(ObstacleType obstacleType)

//Get the prefab corresponding to the given obstacle type
1 reference
private GameObject GetPrefabByType(ObstacleType obstacleType)

//Return an object to the pool by deactivating it
0 references
public void ReturnPooledObject(GameObject obj)
```

4. template method

In abstract class stone, there are some fixed methods with non-fixed methods(abstract).

```
public void SetNickName(String nickName)
{
    this.nickName = nickName;
}

public void ChangeState(IStoneState _state)
{
    this.state.ExitState();
    this.state = _state;
    this.state.EnterState();
}

public abstract void Washing();
public abstract void UpdateHP(float HP);
public abstract void UpdateLoveGage(float Gage);
public abstract void CheckEvolution();
```

디자인 패턴 적용

코드

```
public class GameManager : MonoBehaviour
{
    #region Init

    //Singleton instance
    public static GameManager Instance { get; private set; }

    //References to various managers in the game
    public ControllerManager _controller;
    public MinigameManager _minigame;
    public LogInManager _logIn;
    public CollectingBookManager _book;
    public SearchingUserManager _search;

    //Player information
    public string id;
    public string nickname;
    private bool minigameEnter = true;

    public bool MinigameEnter { get { return minigameEnter; } private set { minigameEnter = value; }}

    //Input, item and stone managers
    private InputManager _input;
    private ItemManager _item;
    [SerializeField]
    private StoneManager _stone;

    //Public access to input,stone and item managers
    public static InputManager Input { get { return Instance._input; } }
    public static StoneManager Stone { get { return Instance._stone; } }
    public static ItemManager Item { get { return Instance._item; } }
```

게임 매니저를 다른 매니저들을 관리하는 느낌으로 생각했습니다. 싱글톤 패턴을 사용했습니다. 지금 생각해보면 다른 매니저를 사용하기 위해서 굳이 게임 매니저에 접근해야 하는 이 구조가 종속성이 커서 별로인 것 같습니다. 때문에 최근 코드를 짤 때는 기능별로 매니저를 독립시켜 종속성을 최대한 감소시키려 노력합니다.

```
void Update()
{
    _input.OnUpdate();
}
```

각 매니저는 MonoBehaviour를 가능한한 상속하지 않았으며, 대신 게임매니저에서 Update()와 Start()함수를 대신 실행해 주었습니다. 경우에 따라 가독성 면에서 이점이 있다고 보여집니다, 지금 코드에서는 너무 억지로 사용하는 느낌이 듭니다.

코드

```
public void SetController(MAP_TYPE map)
{
    if (_button.BackToSauna != null)
    {
        _button.BackToSauna.onClick.RemoveAllListeners();
        _button.BackToSauna.onClick.AddListener(() => GameManager.Instance.ChangeMap(MAP_TYPE.Sauna));
    }

    if (map.Equals(MAP_TYPE.MainTitle))
    {
        _button.signUp.onClick.RemoveAllListeners();
        _button.logIn.onClick.RemoveAllListeners();
        _button.logOut.onClick.RemoveAllListeners();
        _button.gameStart.onClick.RemoveAllListeners();
        _button.quit.onClick.RemoveAllListeners();

        _button.signUp.onClick.AddListener(GameManager.Instance._logIn.SignUp);
        _button.logIn.onClick.AddListener(GameManager.Instance._logIn.LogIn);
        _button.logOut.onClick.AddListener(GameManager.Instance._logIn.LogOut);
        _button.gameStart.onClick.AddListener(GameManager.Instance._logIn.UpdateButtonListener);
        //_button.gameStart.onClick.AddListener(() => GameManager.Instance.ChangeMap(MAP_TYPE.Sauna));
        _button.quit.onClick.AddListener(GameManager.Instance.GameQuit);
    }
```

유니티의 경우 DontDestroy처리 된 오브젝트가 씬을 넘어갔을 때, 인스펙터에 할당된 값들이 초기화됩니다.

해당 게임에서는 게임 매니저를 싱글톤으로서 씬이 넘어가도 파괴되지 않으며, 버튼과 UI를 여기서 관리했기에 씬이 넘어가면 초기화되는 문제가 발생했습니다.

그래서 생각한 것이 ControllerManager를 각 씬마다 독립적으로 배치한 후, 그 씬이 시작될 때 해당 ControllerManager를 게임매니저에 할당하도록 하는 것이었습니다.

지금 생각해보면 불필요한 종속성을 띄고, 가독성도 별로며, 클라이언트 측에서도 할 일이 많아 참 별로인 것 같습니다.

이걸 굳이 매니저로 관리하려 하지 말고 독립적으로 했으면 훨씬 좋았을 것이다.

코드

```
[Serializable]
public abstract class Stone
{
```

전략 패턴을 사용하기 위한 시도로 돌 class를 구상했다.

물론, 지금 봤을 때 이 쓰임새가 전략 패턴과는 사뭇 느낌이 다르다. 돌이 어떤 무기를 사용하는지에 대한 전략이 아니라, 어떤 돌을 사용할지에 대한 전략이라는 느낌이다.

Unity 프로그래밍이 거의 처음이라 클래스를 생성하고 그걸 유니티 에디터에서 어떻게 사용할지 헤매는 바람에 난잡해졌다.

```
private new void OnMouseDown()
{
    base.MouseDown();
    if (GameManager.Instance.nowMap == MAP_TYPE.Sauna)
    {
        stone.UpdateLoveGage(20F);
    }
}
```

때문에 외부 스크립트에서 stone을 호출해 전략에 따른 함수를 호출하는 것이 아니라, 스톤 클래스 내부에서 독립적으로 함수를 호출하는 기이한 구조가 되어버렸다.

참으로 경악스럽다!

이걸 수정하라고 하면 아마 처음부터 짜는 것이 편할 수도 있겠다.

코드

```
IEnumerator GetStoneRequest()
{
    string url = $"{getStoneUrl}/{memberNickName}";
    UnityWebRequest www = UnityWebRequest.Get(url);
    yield return www.SendWebRequest();

    switch (www.result)
    {
        case UnityWebRequest.Result.ConnectionError:
        case UnityWebRequest.Result.DataProcessingError:
            Debug.Log("Error: " + www.error);
            break;
        case UnityWebRequest.Result.ProtocolError:
            Debug.Log("HTTP Error: " + www.error + " Response: " + www.downloadHandler.text);
            break;
        case UnityWebRequest.Result.Success:
            Debug.Log("Success: " + www.downloadHandler.text);
            books = JsonToList(www.downloadHandler.text);
            break;
    }
    yield return null;
}
```

유니티 Networking은 사용해본 적도 있고, 어렵지도 않아서 쉽게 사용할 수 있었다.

이를 이용해 AWS에 등록한 Spring 서버에 요청을 보내고 응답을 받을 수 있다.

이 게임의 경우엔 각 유저마다 어떤 돌을 키웠는지 정보를 서버에 저장한다.

서버에 요청을 보낸 후 응답을 받고, 받은 json 데이터를 list로 변경한 후 게임에서 사용했다.

```
public List<CollectingBook> JsonToList(string jsonString)
{
    jsonString = "{\"stones\":" + jsonString + "}";

    CollectingBookList book = JsonUtility.FromJson<CollectingBookList>(jsonString);

    // Convert the array to a list if needed
    List<CollectingBook> stones = new List<CollectingBook>(book.stones);

    // Now you can use the 'stones' list
    foreach (var stone in stones)
    {
        Debug.Log("Name: " + stone.stoneName + ", Number: " + stone.stoneNumber);
    }

    return stones;
}
```

느낀점

그동안의 경험으로 기획 및 소통 능력은 처음보다 많이 좋아졌습니다.

이 프로젝트는 팀원을 구하는 과정부터 이전과는 달랐는데, 일단 저는 팀장을 하지 않았습니다. 팀장을 하기엔 내 일정이 빡빡하다는 것을 이젠 알고 있었습니다.

대신 팀원을 가장 빠르게 찾는 팀이 나타나기를 기다렸습니다. 보통 팀원을 빠르게 구한다는 것은 팀장이 열정이 있으며 소통에 자신감이 있다는 것을 의미하는 거라고 생각했기 때문입니다. 물론, 대학교 수준에서 그러합니다. 아무튼 그렇게 들어간 팀은 예상대로 좋은 소통 능력과 열정을 가지고 있었습니다. 저는 팀원들과 인사를 나누며 그들이 기획에 있어서 아이디어가 없다는 것을 파악한 후, 아이스 브레이킹과 동시에 자연스레 브레인스토밍으로 이끌었습니다.

팀원들 간의 합의를 이끌어낸 후 나는 본격적인 프로그래밍 작업에 돌입했습니다.

본격적인 Unity C# 프로그래밍은 처음이었는데, 소프트웨어학과 경력 때문인지 어렵진 않았습니다. 또한 그동안 어깨 너머로 배운 게 있어서 완전히 문외한은 아니었습니다.

유니티에서 Manager라는 스크립트들을 사용한다는 건 알았지만 직접 작성해보니까 어떻게 사용해야할지 감이 안 잡혔습니다.

이미 현업에서 일하던 친구들한테 물어보고 결국 내 뜻대로 자유롭게 작성하겠다는 결론을 얻었습니다. (친구는 정답은 없다고 말했습니다.)

그래서 가장 직관적인 방법으로 매니저를 현실의 매니저처럼 역할을 부여하여 사용하기로 했습니다.

동시에 여러가지 디자인 패턴을 적용해보는 연습을 했습니다. 디자인 패턴의 개념을 잘못 이해해서 코드를 잘못 짜거나, 캡슐화가 잘 안 되서 오류가 나는 경우가 허다했습니다. 이전에 소규모 프로그램을 코딩할 때는 몰랐는데, 규모가 커지다보니 왜 코드에 가독성 및 모듈화가 필요한지 단번에 이해하게 되었습니다.

2024

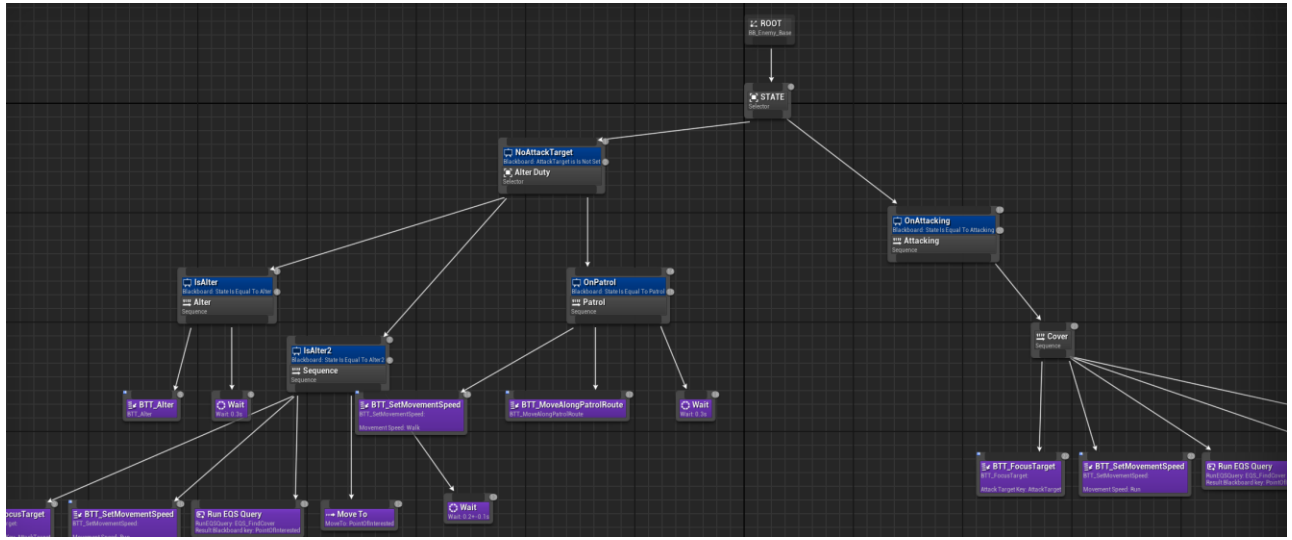
CQB-Training

개요	CQB를 경험하는 FPS 게임
참여	4 Client
역할	Enemy AI 개발
기간	2024.03.~2024.06
환경	Unreal

비고

4학년 Serious Game Project 수업
프로젝트 파일 별첨

내 활동



이 프로젝트에서 나는 적군 AI를 만드는 작업을 담당했다.

언리얼도 처음이고, 심지어 AI는 수업에서 다뤄주지도 않았다.

하지만 소프트웨어학과 4년 경험 덕분인지 API를 금방 다룰 수 있게 되었다.

만드는 게 재밌어서 필요 이상으로 AI를 만들었는데, 그러다가 다른 과제를 놓쳐서 곤혹을 치뤘던 적이 있다.

아무튼 AI를 만드는데 기술적으로 어려운 건 전혀 없었다.

언리얼 API가 너무 뛰어나기도 했고, 팀원이 어떤 강의를 따라 만든 게임도 기반이 탄탄해서 별 문제가 발생하지 않았다.

내 활동

적 NPC는 숨어있으며, 플레이어를 보거나 총 소리를 들으면 움직이기 시작한다. 여기에는 AI Perception을 사용했다.

NPC가 움직일 위치는 EQS를 통해 결정하며, 플레이어의 시야에 닫지 않는 엄폐물 뒤로 이동한다.

NPC가 사격하는 걸 구현하는데 몇 가지 문제점이 발생했었다. 0.1초 만에 탄창을 비우거나, 죽어서도 사격을 하거나, 탄창을 다 썼는데 계속 발사를 시도하는 등의 일이 있었다. 이런 잔 문제점이 EQS, Perception 등에서도 발생했고 이걸 해결하는데 시간을 많이 쓴 것 같다.

내 활동

NPC의 에임이 흔들리게 설계했다. 때문에 낮은 명중률로 플레이어가 쉽게 죽지 않으며 섬광탄에 맞을 경우 에임의 흔들림이 더욱 심해진다.

그리고 패트롤 루틴도 설정할 수 있다. 패트롤 루틴 오브젝트를 만들고 이를 NPC에게 넣으면 경로를 따라 정찰한다.

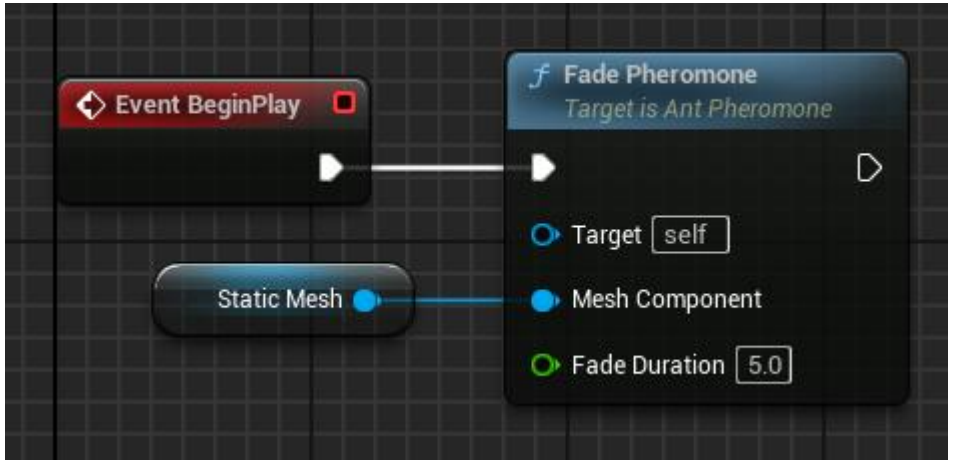
정찰 중에 플레이어를 발견하면 그 즉시 정찰을 종료하고 사격을 개시한다.

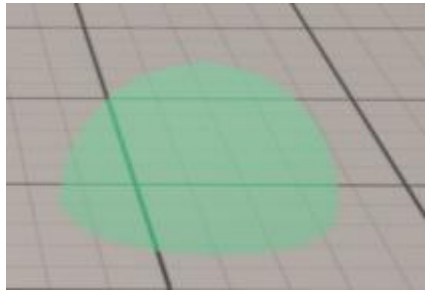
내 활동 (개인적인 연구)

```
AAntAIController::AAntAIController()
{
    PrimaryActorTick.bCanEverTick = true;

    // Create AIPerception and assign it to class variable
    AIPerceptionComponent = CreateDefaultSubobject<UAIPerceptionComponent>(TEXT("AIPerceptionComponent"));

    // Create and set sight-config
    SightConfig = CreateDefaultSubobject<UAISenseConfig_Sight>(TEXT("SightConfig"));
    SightConfig->SightRadius = 600.0f; // 600cm(6m)
    SightConfig->LoseSightRadius = 660.0f;
    SightConfig->PeripheralVisionAngleDegrees = 70.0f;
    SightConfig->SetMaxAge(5.0f);

    SightConfig->DetectionByAffiliation.bDetectEnemies = true;
    SightConfig->DetectionByAffiliation.bDetectFriendlies = true;
    SightConfig->DetectionByAffiliation.bDetectNeutrals = true;

    // Assign sight-config to AIPerceoptionComponent
    AIPerceptionComponent->ConfigureSense(*SightConfig);
    AIPerceptionComponent->SetDominantSense(SightConfig->GetSenseImplementation());

    // Set delegate that called whenever Perception updated
    AIPerceptionComponent->OnPerceptionUpdated.AddDynamic(this, &AAntAIController::OnPerceptionUpdated);
}
```

```
void AAnt_Pheromone::FadePheromone(UStaticMeshComponent* MeshComponent, float FadeDuration)
{
    if (MeshComponent)
    {
        UMaterialInterface* Material = MeshComponent->GetMaterial(0);

        if (Material)
        {
            UMaterialInstanceDynamic* DynamicMaterial = UMaterialInstanceDynamic::Create(Material, this);
            MeshComponent->SetMaterial(0, DynamicMaterial);

            if (DynamicMaterial)
            {
                const float TimerInterval = 0.1f;
                ElapsedTime = 0.0f;

                GetWorld()->GetTimerManager().SetTimer(
                    FadeTimerHandle,
                    [this, DynamicMaterial, FadeDuration, TimerInterval]()
                    {
                        // 시간이 흐름에 따라 투명도를 조절
                        ElapsedTime += TimerInterval;
                        float Alpha = FMath::Clamp(ElapsedTime / FadeDuration, 0.0f, 1.0f);
                        // 머티리얼의 "Opacity" 파라미터 값을 조정 (0: 투명, 1: 불투명)
                        DynamicMaterial->SetScalarParameterValue(TEXT("Opacity"), 1.0f - Alpha);
                        // 투명화가 완료되면 액터를 파괴하고 타이머를 정지
                        if (ElapsedTime >= FadeDuration)
                        {
                            Destroy(); // 올바른 타이머 정지 방식
                            GetWorld()->GetTimerManager().ClearTimer(FadeTimerHandle);
                        }
                    },
                    TimerInterval,
                    true); // true로 반복 실행
            }
        }
    else

        UE_LOG(LogTemp, Warning, TEXT("MeshComponent is not set in Blueprint."));
```

블루프린터로 일일이 편집을 하는 것보다, 스크립트를 상속받는 것이 더욱 '추상적'이라 생각해서 스크립트로 기본 셋팅을 해주는 코드를 작성해봤다.
당시 개미의 움직임에 관심이 많아 이를 구현해보려는 시도의 흔적으로 이름이 AntAI이다.





개미 움직임의 중추인 페로몬이다. 만약 내가 알고리즘을 완성시켰다면, 적군 AI 는 이를 중심으로 움직이게 될 것이다.

페로몬은 시간이 지나며 사라지며, C++로 작성한 블루프린트 노드에서 변수를 편집할 수 있다. 시간이 부족해서 여기까지만 시도해보고 그만뒀다.

느낀점

언리얼을 다루는 것은 처음이고, 개발자만 모인 팀에서 프로젝트를 한 것도 처음이다.

초기 프로젝트 설계 단계에서 이야기를 나누는데 토의가 너무 답답했던 기억이 있다. 어떤 게임을 만들 것인지에 대한 이야기가 아니라, 어떤 시스템을 구축할지에 대한 얘기만 오갔기 때문이다. 항상 이야기부터 창작을 시작하는 나로선 시스템 설계만 주구장창 하는 그 자리가 너무 무의미하게 느껴졌었다. 그 당시 나는 그 자리를 이해하지 못 하고 속으로 화를 내며 조용히 있었다. 조용히 있었던 이유는 내 생각이 틀렸을 수도 있다는 가능성을 검토 중이어서 그랬다. 얌전히 토론을 끝내고 집으로 돌아오는 길에 그 상황을 계속 되짚어 봤다. 동기한테도 그 상황에 대해 설명하며 내가 무엇을 놓쳤는지에 대한 조언을 구했다. 내가 내린 결론은 틀린 사람은 없고, 그냥 내 성향이 안 맞았던 것 뿐이라는 것이다. 이러다가 좋은 게임을 못 만들 것 같아서 내 스스로를 팀 분위기에 맞췄고, 결국 화기애애하게 프로젝트를 진행하고 좋은 게임을 만들 수 있었다.

팀원들은 동기 중에서도 실력 있는 개발자들이었기 때문에 색다른 모습을 관찰할 수 있었다. 그들이 게임에 적용할 기술에 대해 서로의 논리를 펼치며 설명하고 공유하는 열정 있는 모습이 너무 보기 좋았다. 약간 코드에 심취한 덕후들 같았다. 한 팀원은 개발이 너무 재밌다고 일주일만에 대부분의 시스템을 구현해 왔는데 나는 그 때 진짜 엄청 놀랐었다. 대체 그럴 시간이 어디 있던 걸까? 나는 그 모습을 펠월드를 삼일동안 50시간 하던 내 모습과 겹친다는 것을 알았고, 개발에 대핸 내 열정을 더 키우기로 마음 먹었었다.

언리얼 엔진과 AI관련 API를 이해하는데 약 한 두 주 걸렸다. 대학생이다 보니 실질적인 시간은 더 적었지만, 유튜브 강의를 보며 빠르게 숙달할 수 있었다. 그 후는 만들어야 하는 걸 만들고 내가 만들고 싶은 기능들을 넣었다. 특히 NPC를 자연스럽게 하는데 시간을 쏟았다. 그러다가 꽤 그럴듯한 NPC가 탄생했다.

당시 언리얼 강의를 진행하던 교수님은 외국분이셨다. 매우 열정적이셔서 내가 물어본 대부분의 것을 10분 정도 설명해 주셨고, 도저히 해결 못하던 오류들은 직접 디버깅까지 하셨다. 기억에 남는 질문 중 하나가, NPC의 EQS가 문 너머 위치를 잡지 못 한다는 것이었다. 교수님도 같이 고민해주시고 자료까지 찾으셨는데 마땅한 해결책이 나오지 않았다. 그러다가 문득 EQS에서 문 오브젝트 자체를 무시하도록 레이어 설정 비슷한 걸 하는 아이디어가 떠올랐고, 교수님은 좋은 아이디어라며 칭찬하셨다. 지식의 보고인 교수님께서 나같은 학생과 함께 어떤 문제를 진지하게 탐구하는 모습을 보고, 이런 분과 함께 일한다면 무척 즐거울 것 같다는 생각이 들며 본받겠다는 생각을 했었다.

2021

Devil’s Winery

개요	악마와 계약해 와인을 만들고 판매하는 게임
참여	2 Client, 2 Game-Designer, 2 Art
역할	Project Manager / Main Game Designer
기간	2022.07~2023.01
환경	Unity, Trello, Slack, Word, Excel, Google Drive

비고

아주대학교 창업동아리 / 파란학기

후기(Link)

2021년 한국게임학회 추계 학술발표대회 논문집 251P

프로젝트 파일 / 문서 별첨

내 활동

전문지식을 소재로 하는 게임 개발 사례

유승민○, 진연규, 김혜경, 윤하연, 손효식, 이효원, 김명수*
아주대학교
kcgs@kcgs.or.kr, ygjin00@ajou.ac.kr, hyegyeong@ajou.ac.kr, kcgpa@hanmail.net
copomos00@ajou.ac.kr, jjingha0407@ajou.ac.kr, hanamaru3682@ajou.ac.kr,
saku1003@ajou.ac.kr, mike0905@ajou.ac.kr

Case of game development based on expert knowledge

Seng-Min You○, Yeon-Gyu Jin, Hye-Gyeong Kim, Hay-Eon Yoon, Hyo-Sik Son,
Hyo-Won Lee, Myeong-Su Gim
Ajou University

요 약

본 논문은 전문지식이 소재인 게임의 기획 및 구현에 관한 내용이다. 구체적으로 와인 전문지식을 다룬다. 논문 전반에 걸쳐 전문지식의 습득과 활용 과정을 기술했으며, 또한 전문지식을 소재로 삼았을 때 생기는 문제점을 소개하고 그에 대한 방안법을 제시했다. 이 내용이 게임업계에 도움이 될 수 있기를 희망한다.

1. 서 론

전문지식을 소재로 게임을 만드는 것은 어려운 일이다. 이에 본 논문은 와인 전문지식을 소재로 게임을 만들며 그 과정을 정리해 노하우를 제공하고자 한다.

2. 연구 내용

3.1 전문지식 선정

(1) 방향성

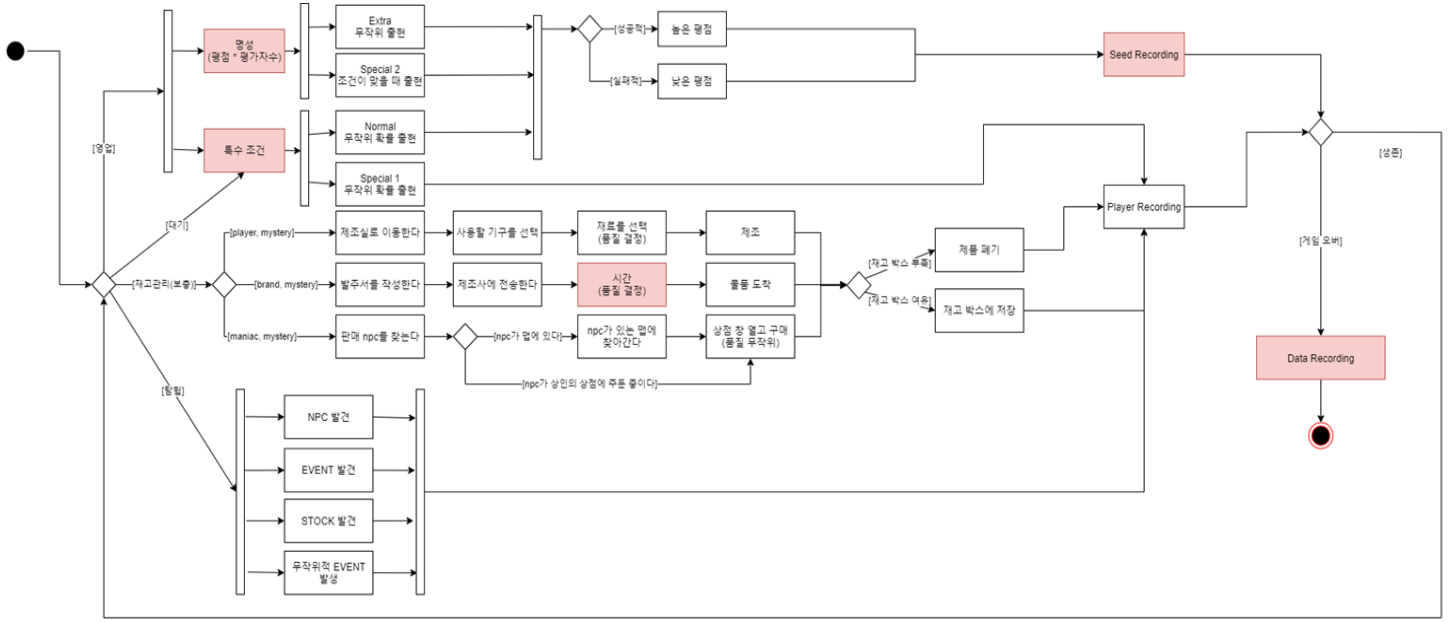
전문지식을 선정하는데, 어떤 전문지식을 습득하고 어떻게 사용할 것인지 방향성을 정한다. 전문지식의 분량은 거대하기 때문에 방향성이 뚜렷하지 않으면 불필요한 지식까지 습득하는 비효율이 발생한다. 본 논문의에서 다루는 방향성은 아랫 문단과 같다.

현대문화에서 와인이란 비즈니스 미팅부터 가족 식사까지 다양한 자리의 분위기를 고급스럽게 하는 식품이다. 많은 사람들이 식사 자리에 와인을 채용하지만 그럼에도 와인의 깊은 문화를 이해하기란 힘들다. 본 게임은 이러한 상황 속에서 사회의 와인 문화 수준을 증진시키는 것에 목적을 두고 있다.

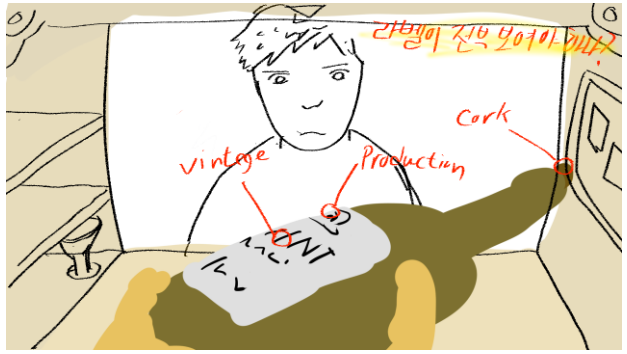
(2) 타겟층 페르소나 분석

방향성을 확립했을 때 이에 개발자들이 흥미를 느낄 수 있어야 한다. 개발자들이 흥미를 느끼지 못하면 동기 부여가 부족해 개발이 힘들어질 수 있기 때문이다. 따라서 타겟층을 정하고 개발자들이 범주에 속하는지 확인해야 한다. 이를 위해 먼저 타겟층 페르소나를 분석한다.

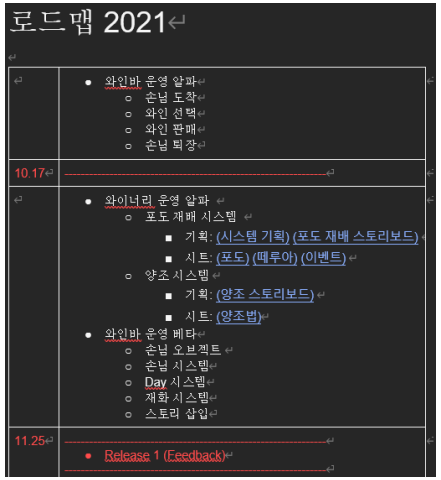
한국게임학회 학술대회 투고



기획



기획



로드맵 2021

- 와인바 운영 알파
  - 손님 도착
  - 와인 선택
  - 와인 판매
  - 손님 퇴장

10.17

- 와인너리 운영 알파
  - 포도 재배 시스템
    - 기획: (시스템 기획) (포도 재배 스토리보드)
    - 시트: (포도) (떼루아) (이벤트)
  - 양조 시스템
    - 기획: (양조 스토리보드)
    - 시트: (양조법)
- 와인바 운영 베타
  - 손님 오브젝트
  - 손님 시스템
  - Day 시스템
  - 재화 시스템
  - 스토리 삽입

11.25

- Release 1 (Feedback)

PM

느낀점

이 프로젝트는 기념비적인 제 첫 실패작입니다.

아무것도 모른 채 무작정 달려들었으니 당연한 결과였습니다. 어쩌면 이는 대학생의 특권일 수도 있겠습니다.

열심히 한다고 하긴 했습니다. 다만, 열심히 하는 것과 잘 하는 것은 별개였습니다.

제가 특히 재능이 없던 부분은 팀을 운용하는 능력이었습니다.

소학회의 대단한 친구들을 모아 팀을 꾸렸으나, 제 운영이 너무나 미숙해서 제대로 돌아가질 않았습니다.

때문에 팀원들은 빠르게 흥미를 잃고, 덩달아 게임의 생명도 꺼져갔습니다.

그러다가 그저 학점만 받을 정도로 최소로 일하게 되는 지경에 다다랐습니다.

전부 끝나고 이 여정을 되돌아봤습니다.

저는 기획을 자꾸 번복했고, 충분히 매력적으로 만들지 못했습니다.

일정 잡는 능력도 최악이라 제때 되는 것이 없었습니다.

무엇보다 저는 왜 팀원들이 대부분 취업 준비생 신분이란 것을 간과하고 팀을 꾸렸었던 걸까요? 팀원들에겐 여유가 없었습니다.

이 모든 실패는 지금까지도 큰 교훈이 되고 있습니다. 팀원의 입장을 이해하고 그들의 적극적인 참여를 끌어내는 방법을 익혀가고, 기획을 번복하는 일이 없게 하게 되었습니다.

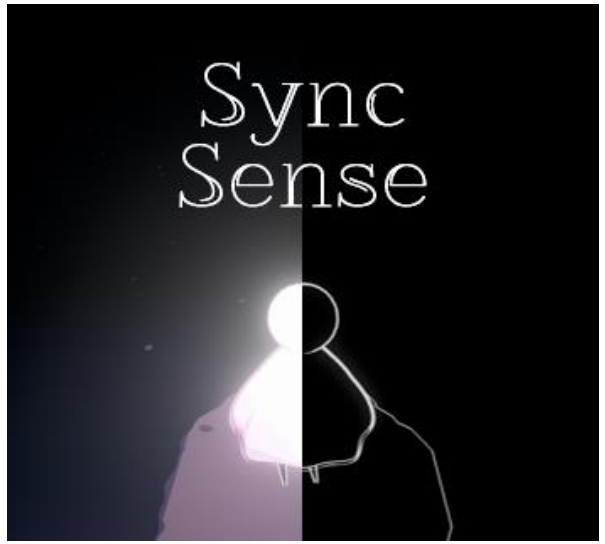
또한 게임 개발이 고행이라는 것을 처절하게 느낀 첫 순간이었습니다. 그럼에도 제 열정은 전혀 꺾이지 않았습니다. 배울 수 있는 기회를 얻었다는 것이 그저 기뻤습니다.

가끔 팀원들과 술자리를 가지며 회포를 풉니다. 그럴 때마다 이 프로젝트를 진행하며 있었던 고행과 다툼이 오히려 결속이 계기가 될 수 있다는 걸 느낍니다.

이러한 경험은 소프트웨어학과에서의 경험과 결합하여, 게임을 위한 코드를 작성할 수 있는 포석이 되어줍니다.

2023

Sync Sense



개요	하나의 감각을 둘로 나눠 퍼즐을 해결한다
참여	1 Programmer, 2 Game-Designer
역할	기획, 맵 배치, 퍼즐 프로그래밍
기간	2023.07~2023.12
환경	Unity, Word, Google Drive

비고

2023 아주대학교 인디게임제작 수업

2023 GEEKS 게임부문 대상

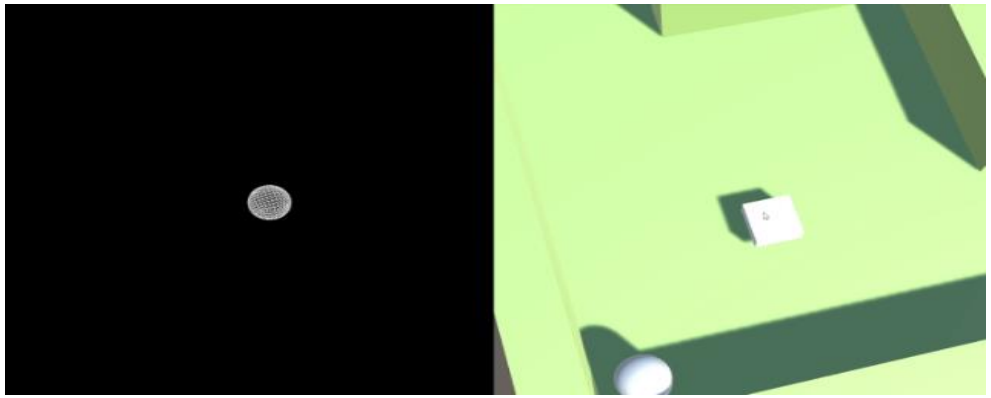
2024 GIGDC 제작부문 동상

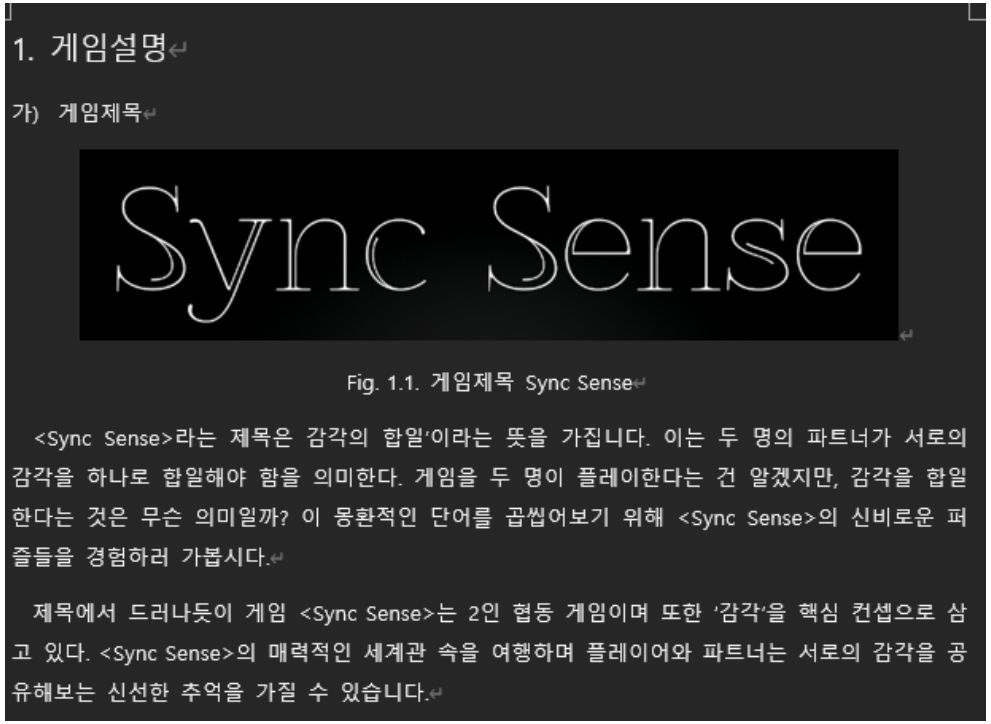
후기(Link)

기획 문서 별첨

유니티 프로그래밍의 시작

내 활동





1. 게임설명

가) 게임제목

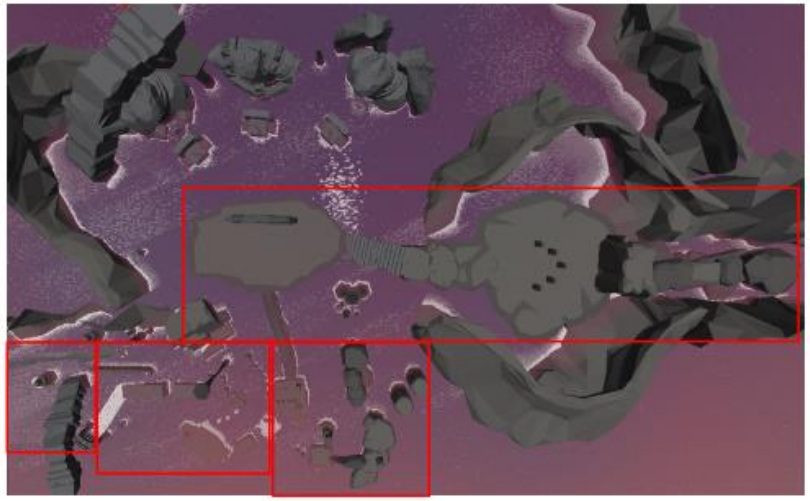
Sync Sense

Fig. 1.1. 게임제목 Sync Sense

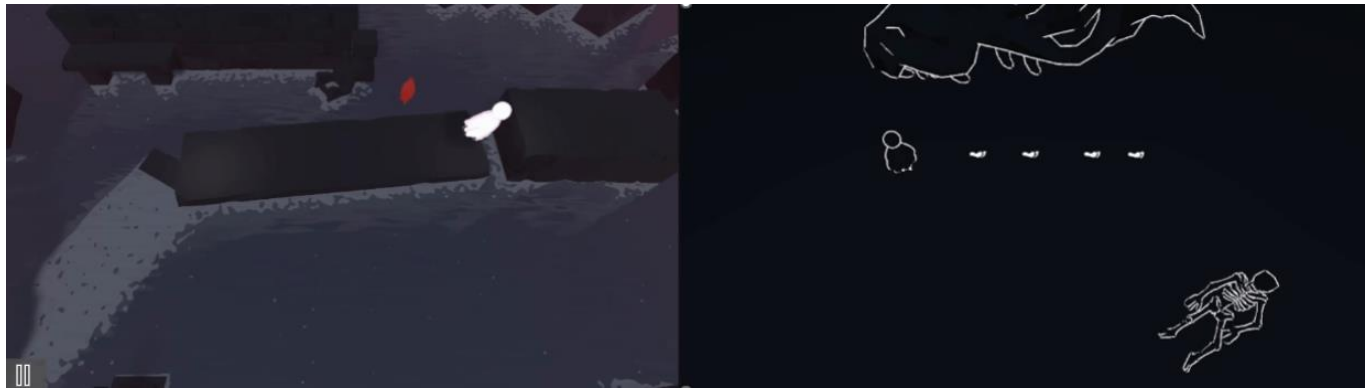
<Sync Sense>라는 제목은 감각의 합일'이라는 뜻을 가집니다. 이는 두 명의 파트너가 서로의 감각을 하나로 합일해야 함을 의미한다. 게임을 두 명이 플레이한다는 건 알겠지만, 감각을 합일한다는 것은 무슨 의미일까? 이 몽환적인 단어를 곱씹어보기 위해 <Sync Sense>의 신비로운 퍼즐들을 경험하러 가봅시다.

제목에서 드러나듯이 게임 <Sync Sense>는 2인 협동 게임이며 또한 '감각'을 핵심 컨셉으로 삼고 있다. <Sync Sense>의 매력적인 세계관 속을 여행하며 플레이어와 파트너는 서로의 감각을 공유해보는 신선한 추억을 가질 수 있습니다.

기획



맵 배치



퍼즐 멀티 테스트 프로그래밍

느낀점

Syne Sense는 대학교 수업에서 제가 아이디어를 제안하고 팀을 꾸려서 시작된 프로젝트입니다.

Devil's Winery에서 행했던 실패를 교훈삼아 좋은 기획을 만들고 활발한 팀 활동을 할 수 있었습니다.

다만, 기획의 경우 팀원들이 너무 강압적으로 의견을 제시하던 터라 제 생각을 많이 꺾을 수 밖에 없었습니다. 마음 속으론 굉장히 답답했지만, 팀원의 사기증진을 위해서라면 감내할 수 있었습니다.

따라서 저는 팀원들의 의견을 최대한 매력 있게 섞어내는 일을 했습니다. 동시에 저희 팀의 유일 프로그래머 옆에 딱 붙어서 친구가 지치지 않을 수 있도록 같이 작업하며 분위기를 높였습니다.

그러다 문제가 발생했습니다.

개발이 더 이상 기획을 따라오지 못했습니다.

애초에 개발자 친구는 기획을 따라 개발하지 않고, Netcode를 공부하는 샘 멀티 플레이 환경을 구축하는 데 전력을 다하고 있었습니다.

그렇다고 저는 '갑'이 아니기에 친구가 하고 싶다는 걸 질책할 순 없었습니다.

그래서 저는 프로그래밍을 시작했습니다.

소프트웨어학과에서 여러 학문을 3년 동안 배운 덕분에, Unity C# 프로그래밍이 낯설진 않았습니다.

유니티 스크립팅 API 중 하나인 Netcode도 쉽게 사용할 수 있었습니다. 마침 다른 과목에서 RPC를 배웠던 터라 동작 원리가 어렵진 않았습니다.

긴 여정을 마치고 프로젝트가 중단된 지금 시점에선 상당히 부담스러웠던 프로젝트였다고 회상합니다.

너무나 강압적인 기획 제안 방식과, 게임 개발 경험이 너무 많은 개발자 팀원을 따라가기 힘들었던 기억 때문인 것 같습니다.

그래서인지 이 게임에는 애정이 생기질 않았습니다. 상을 타고, 회사에서 명함을 주고 가도 제 것 같지가 않았습니다.

그래서 다짐했습니다. 개발하는 게임을 제 것처럼 생각하기 위해서, 일취월장 하겠다고...

기타 프로젝트

2013

스타 유즈맵

참여	1명
역할	All
기간	???
비고	.

내 활동

배틀 오브 호러매니악이라는 이름의 전투 유즈맵. 당시 활동하던 카페의 이름을 따왔다. 각 플레이어는 영웅을 선택한 후 한정된 자원으로 유닛을 뽑아 적 게이트웨이를 파괴하면 된다.'

특별한 개성은 없지만 어렸을 때였음에도 밸런싱을 꽤 신경 쓴 흔적이 보인다. 변신하면 몇 초간 무적이 되는 유닛이지만 변신이 풀리면 피가 1이 돼서 조심해야 된다거나.

순수한 게임에 대한 열정으로 만들었던 만큼 오랫동안 기억하고 싶다.

인페스티드 테란 컨트롤이라는 유즈맵도 만들었었다. 인페스티드 테란을 이용해 몰려오는 웨이브를 섬멸해야 하며, 살아남은 유닛은 다음 웨이브로 보존된다.

업그레이드 및 뽑기같은 다양한 선택지도 마련했었다. 때문에 내가 생각치 못한 플레이로 게임을 깨는 플레이어가 종종 있었으며, 이는 날 굉장히 즐겁게 했다.

간단한 RPG도 하나 만들고 유저들과 함께 플레이하며 반응을 살피는 것을 즐겼던 기억이 있다. 그러다가 누군가 혹평을 하고 게임을 종료해버리면 큰 상처를 받았지만, 그걸 피드백 삼아 맵을 업데이트 했었다.

마지막 유즈맵은 능력 배틀 컨셉이었다. 당시 내가 멋있다고 생각했던 것들을 전부 때려박기 시작했다. PVP의 밸런싱은 굉장히 까다로운 것이였고, 내 맵의 밸런싱은 엉망이었던 걸로 기억한다.

2015

RPG MAKER

참여 1명

역할 All

기간 ???

비고

RPG 메이커는 제한적인 스타크래프트 유즈맵에서 벗어나 좀 더 거대한 게임을 만들고 싶어서 접하게 된 툴이다. 이것 또한 스타크래프트 유즈맵을 접했을 때처럼 아무런 지식 없이 그냥 만들기 시작했고, 어쩌다가 원하는 것을 만들 수 있는 정도가 되었다.

'마지막의 길 ' 이라는 이름의 공포 퍼즐 게임을 만들었다. 한 소년이 길을 걸으며 과거를 회상하는데, 그 내용이 의아스럽고 길의 마지막에는 지옥으로 향하는 문이 있다는 내용이었다. 처음에는 신나서 만들다가 점점 아이디어도 떨어지고 이야기도 난잡해졌다. 그러다가 결국 게임을 망치고 말았다.

큰 볼륨의 게임을 만들기 위해서는 충분한 생각과 시간이 필요하다는 것을 깨닫는 계기가 되었다.

2019

투구: 심해의 모험가

참여 3명

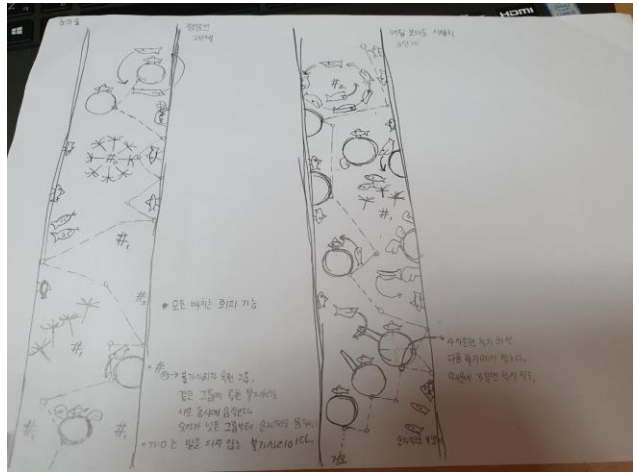
역할 Assist (QA/Pattern Design)

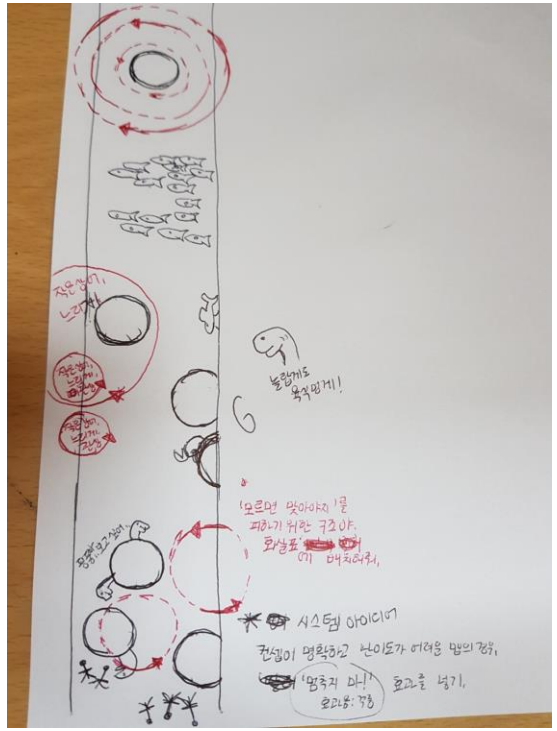
기간 2019.01~2019.12

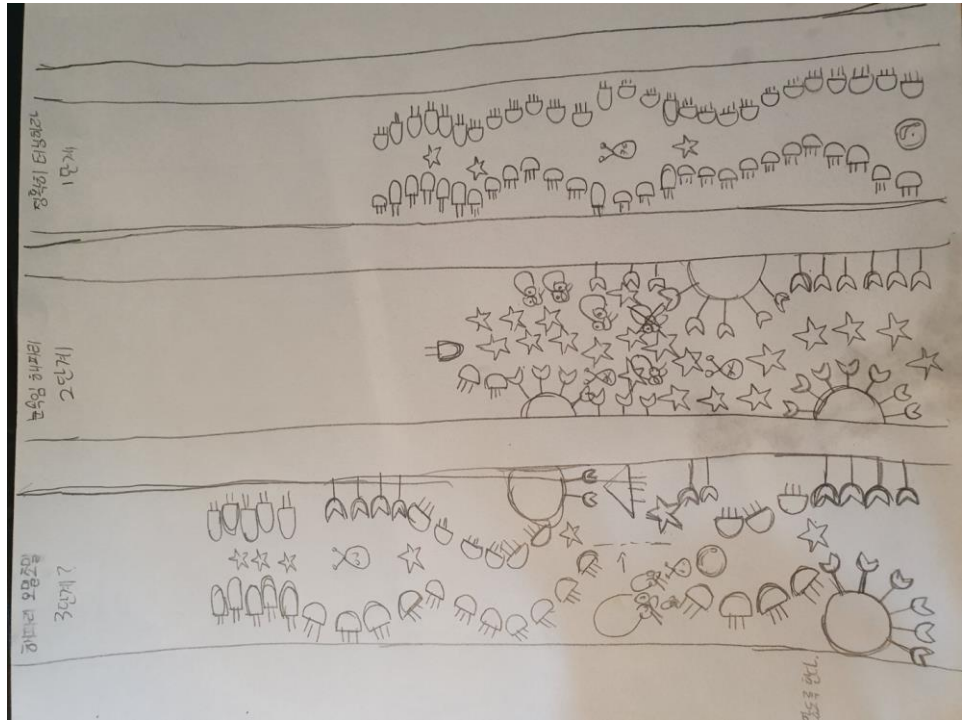
비고 스마일게이트 멤버십 11기

후기(Link)

내 활동







2019

뽀삐 할로윈

참여 6명

역할 Boss Artist

기간 2019.10.31

비고 할로윈 당일치기 게임잼

영상 별첨

내 활동

2019

동물의 술

참여 3명

역할 Game Designer

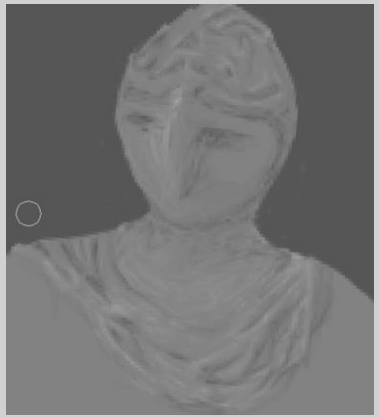
기간 2019.02.04 ~ 2019.02.06

비고 소학회 TML 게임잼

후기(Link)

빌드파일/제작BGM파일 별첨

내 활동



2020

New Age

참여 3명

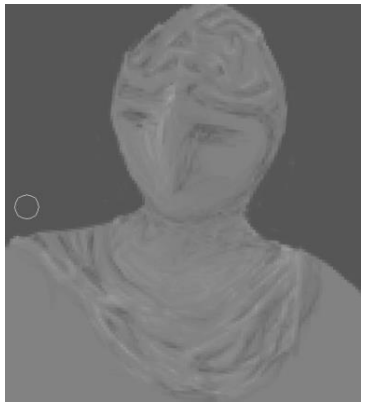
역할 Background Artist

기간 2020.01

비고 후기(Link)

이미지 파일 별첨

내 활동



2020

Infinite breakwall

참여 4명

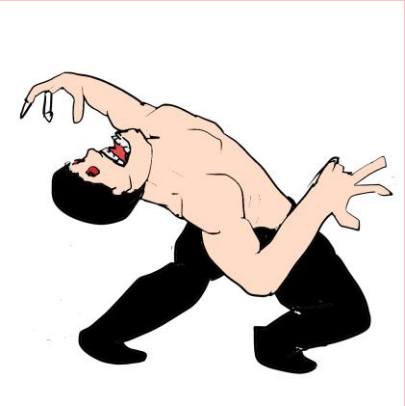
역할 Main Artist

기간 2020.08.01 ~ 2020.08.15

비고 후기(Link)

이미지 파일 별첨

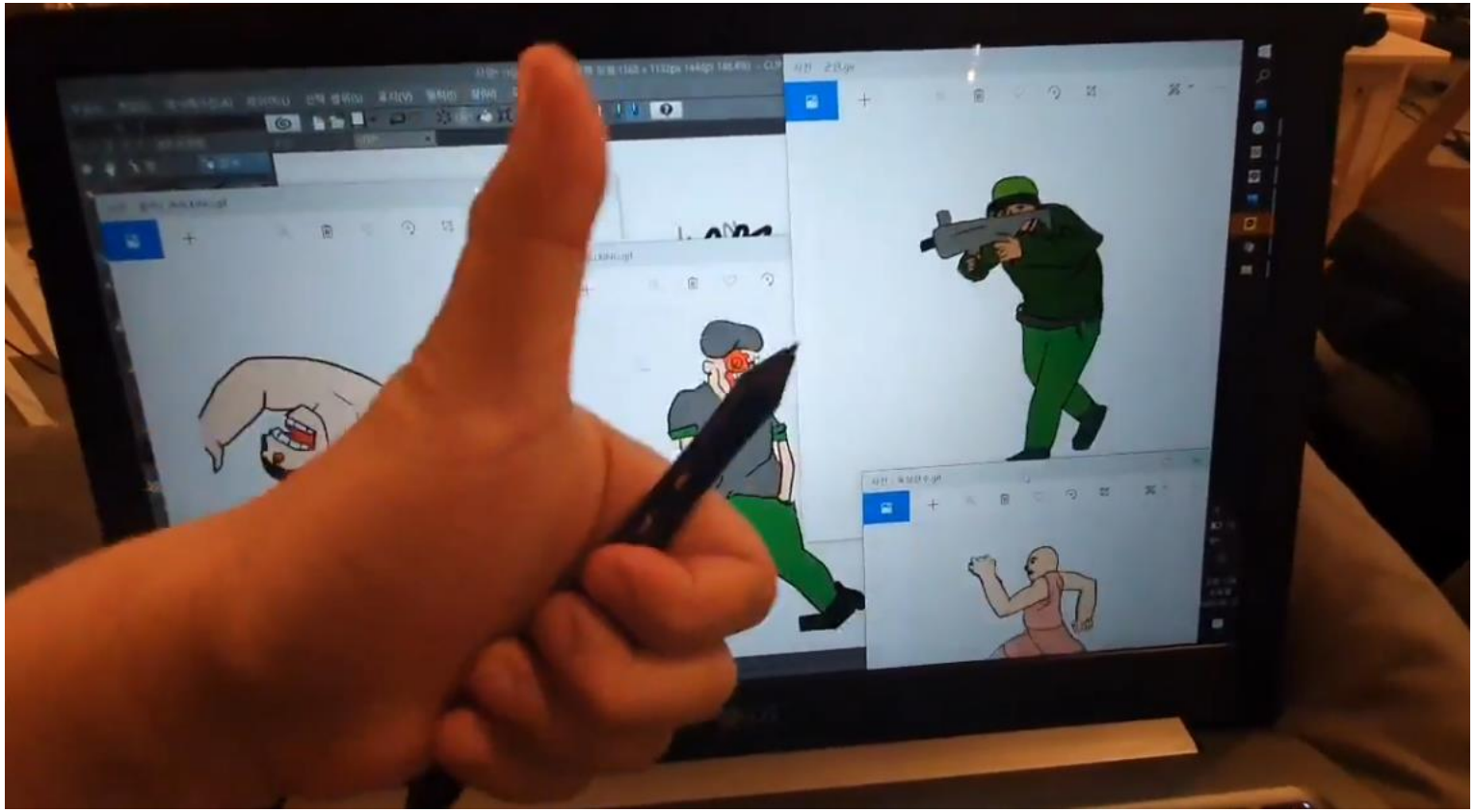
내 활동

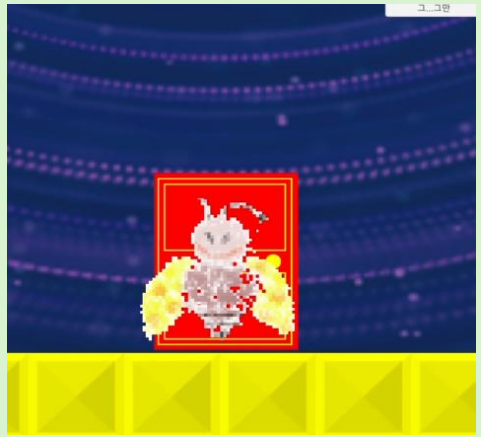


2020

좀비 아포칼립스

참여	4명
역할	Character Artist
기간	2020.08.18 ~ 2020.08.19
비고	소학회 TML 게임잼
	이미지 파일 별첨





2021

BUG

참여 2명

역할 Main Programmer

기간 2021.01.27~2021.01.28

비고 소학회 TML 게임잼

프로젝트 / 빌드 파일 별첨

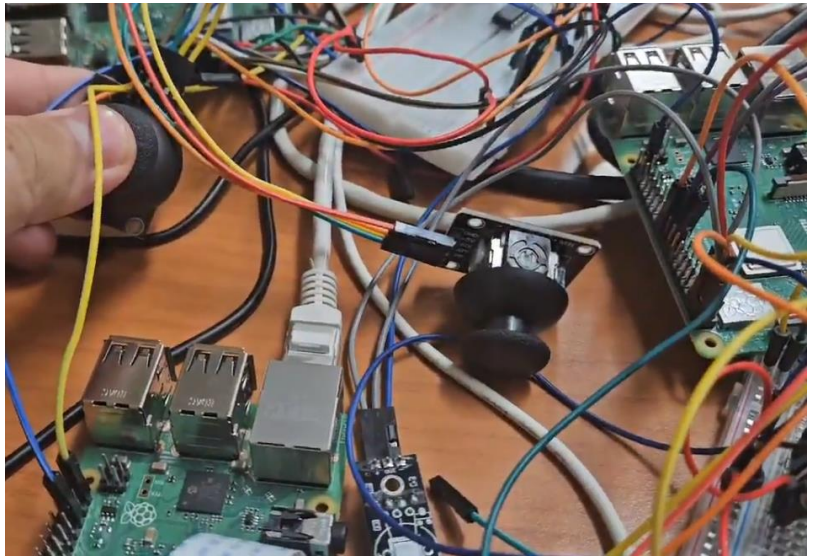
내 활동

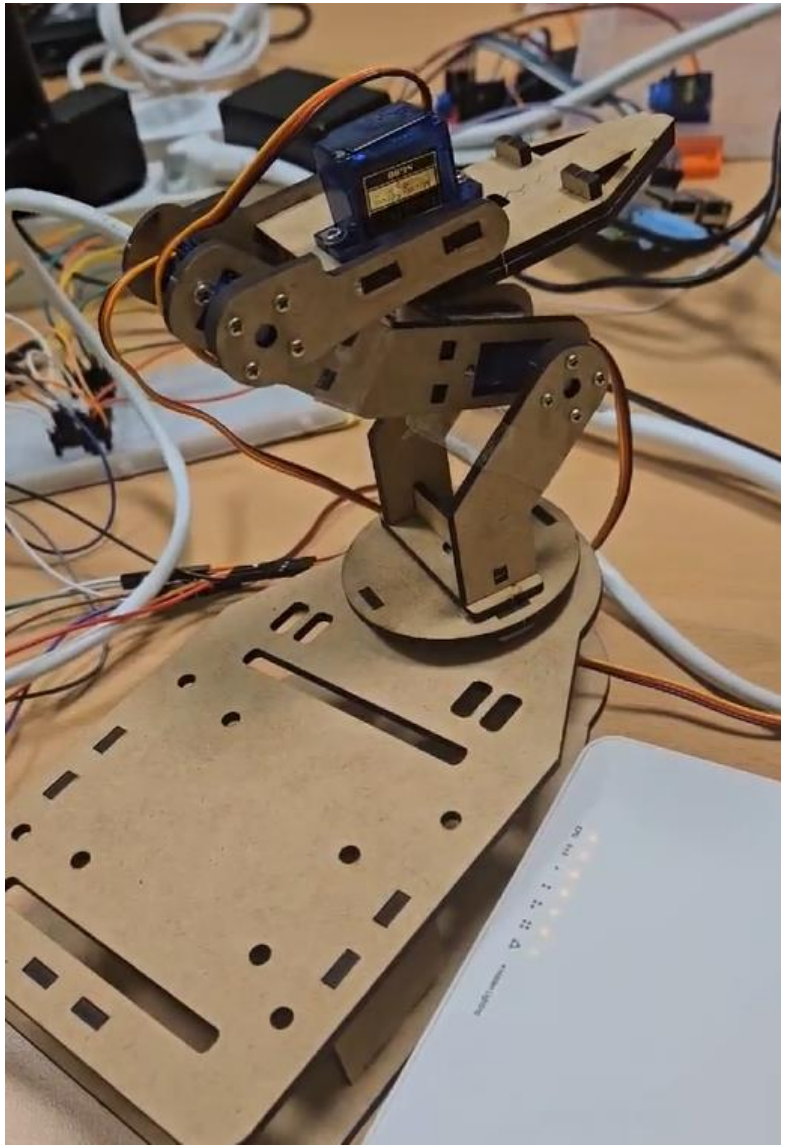
게임
외

2024

라즈베리파이 프로그래밍 & C언어 소켓 통신

개요	라즈베리파이를 필두로 시스템 하나 제작하기
참여	4명
담당	소켓 통신 서버/클라이언트 프로그래밍 & OpenCV
기간	2024.06 – 2024.07
환경	라즈베리파이, C, OpenCV
비고	시연 영상 및 코드 별첨





2023

Twitch Chat Care

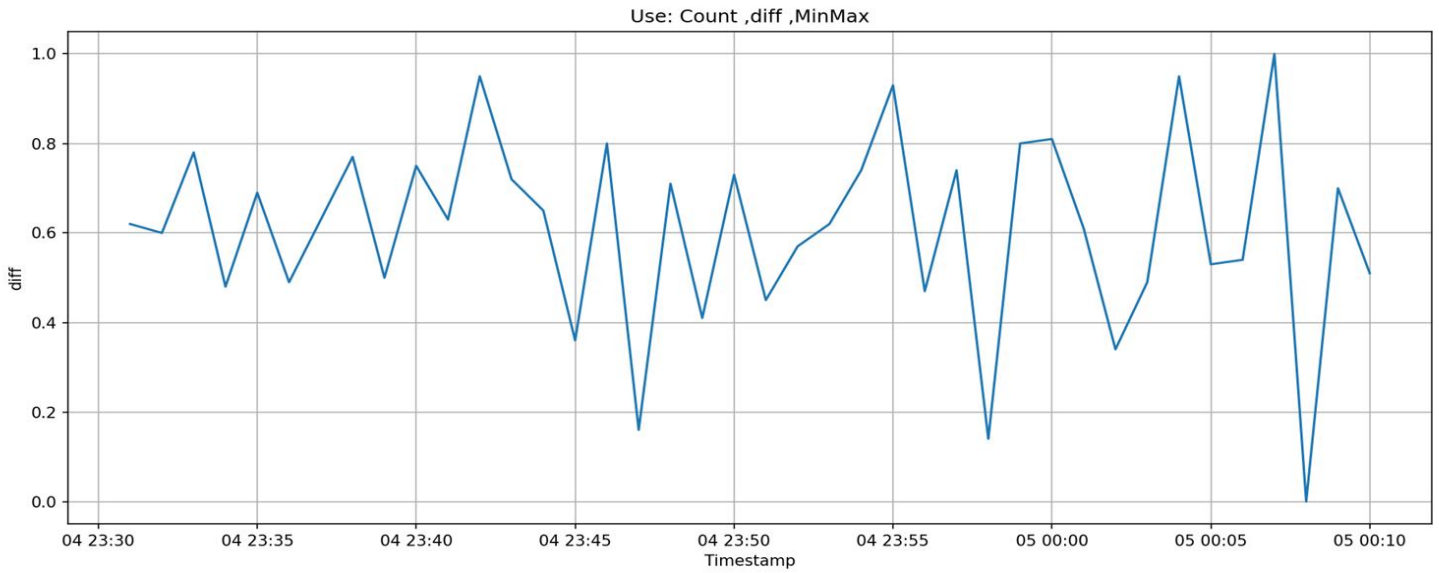
[프로젝트 LINK]

개요	NLP Task, Twitch Chat 분석
참여	3 Programmer
역할	데이터 전 처리
기간	2024.10~2024.12
환경	Python
비고	4학년 AI 집중교육2 수업

[블로그 LINK]

```python
import livenlp as lvn
from livenlp import livechat as lvc
from livenlp import statistics as lvns
```

```python
vibe = [    {'재미' : ['ㅋㅋ']}, {'감탄' : ['캬', '와']},
```



	datetime	score
0	2023-11-13 20:06:36	-0.417533
1	2023-11-13 20:06:46	-0.194444
2	2023-11-13 20:06:48	-0.121126
3	2023-11-13 20:06:52	-0.375863
4	2023-11-13 20:07:49	0.106965
..	...	...
936	2023-11-14 01:51:52	0.794227
937	2023-11-14 01:52:43	-0.691150
938	2023-11-14 01:52:53	-0.251584
939	2023-11-14 01:52:55	-0.230383
940	2023-11-14 01:53:46	-0.691150

2020

SIDIZ 광고

LINK

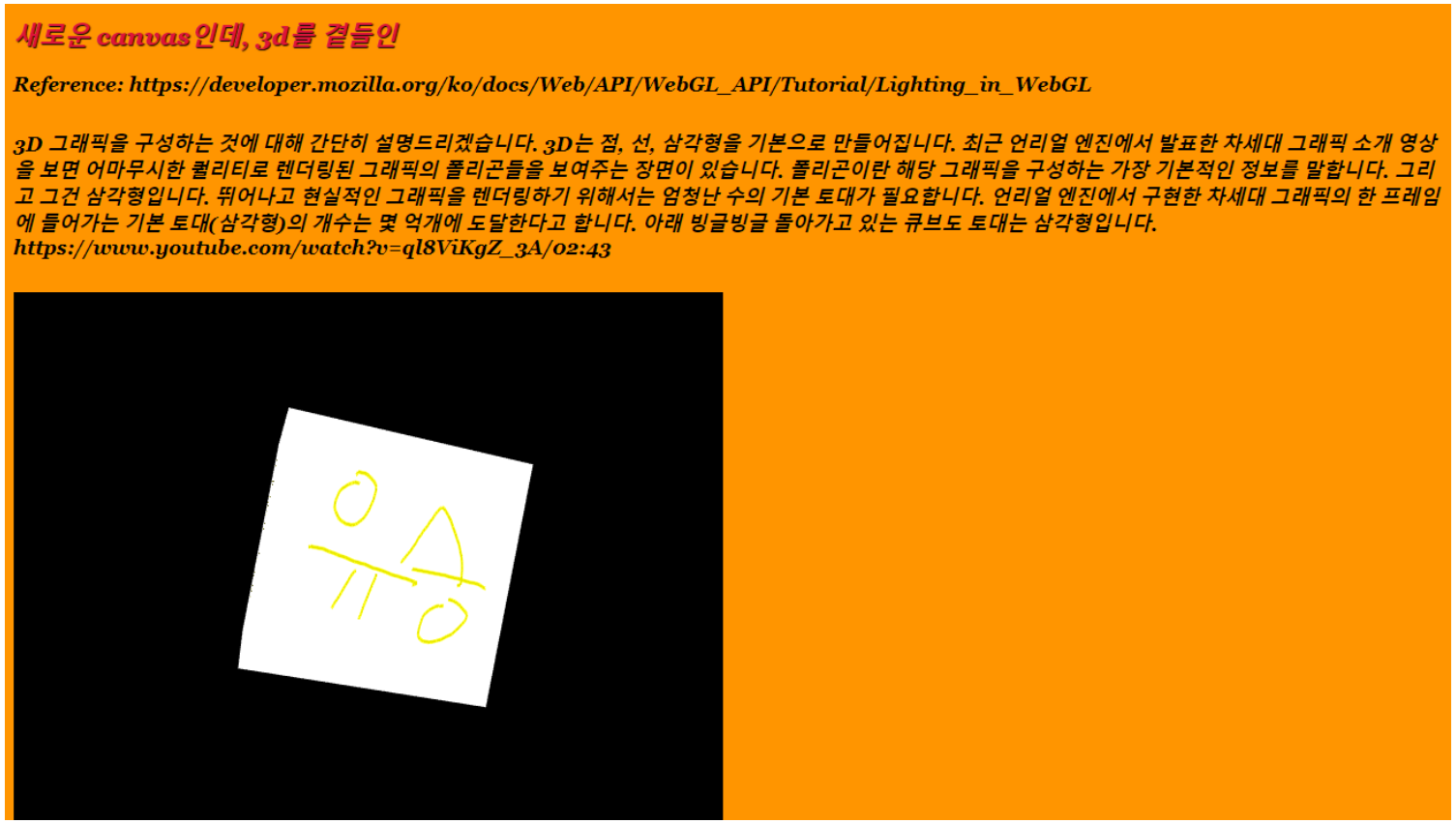
개요	나의 첫 애니메이션. SIDIZ 광고 만들기
참여	개인
기간	2020.04
환경	Clip Studio
비고	2학년 시리우스게임분석 수업

2021

WebGL Tutorial

페이지 LINK

개요	WebGL을 모르는 사람을 위한 자료
참여	개인
기간	2021.07~2021.12
환경	HTML, JS, WebGL
비고	3학년 컴퓨터 그래픽스 수업



2021

Start Craft I Network 분석

[문서 LINK]

개요	Wireshark를 사용해 아무 네트워크나 분석해보라
참여	개인
기간	2021.08
환경	Wireshark
비고	3학년 컴퓨터 네트워크 수업

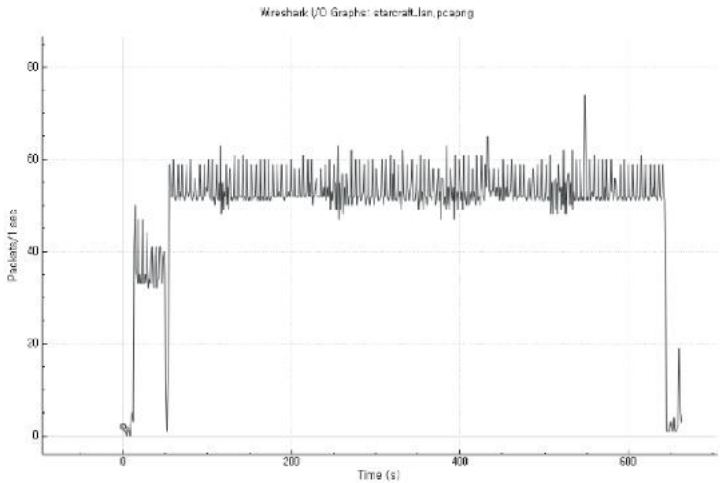
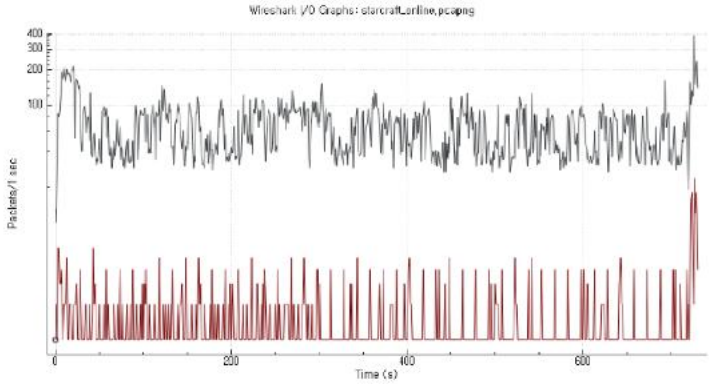


그림 1 <starcraft LAN game I/O Graphs>

그림 1 은 LogMein Hamachi 프로그램을 이용해 가상 사설망 (VPN)을 만들어 게임 starcraft 에서 LAN game 을 플레이했을 때의 I/O Graphs 다. 가상 사설망에서만 packet 을 capture 했으며, 모든 packet 이 가상 사설망 속 친구와 주고받은 것들이다(넷 게 예외가 있을 수는 있다.). LAN game 의 상대는 내 친구다. 관찰 환경의 변화는 LAN server 접속, 친구가 개설한 사설 방에 입장, 게임 플레이, 게임 종료까지의 단계를 거쳤다. LAN server 에 접속했을 때 traffic 이 높다가 게임을 시작하는 순간에 traffic 이 한 순간 급감한다. 그 후 본격적인 게임이 시작되며 이때부터 traffic 이 꾸준하게 높다. 게임이 끝나고 게임 결과창이 나왔을 때 traffic 이 급감하며 이 때 capture 를 중단했다.



2021

머신러닝 AI 레벨 디자인

문서 LINK

개요	머신러닝으로 만든 AI는 어떻게 레벨 디자인을 해야 하는가?
참여	개인
기간	2021.07~2021.10
환경	Unity ML-Agent
비고	3학년 자기주도연구 수업

2.1.3 F1 2021

AI Score 라는 이름의 점수가 0-110 까지 존재한다. 각 점수는 코스 주행 시간과 매칭되어 AI 의 실력이 정해진다.

2.2 알고리즘

2.2.1 유저 적응형 강화학습

강화학습과 난이도를 키워드로 검색한 결과는 대부분 강화학습으로 게임 난이도를 평가하는 것과 게임 속에서 플레이어에 맞춰 에이전트 난이도가 저장되는 것들이다. 대표적으로 게임을 길찾기 문제로 보고 A*알고리즘으로 게임 난이도를 수치로 표시하는 것과, 이분법적으로 표현하기 힘든 애매한 표현들을 퍼지 이론으로 처리하여 플레이어 실력에 맞춰 게임 난이도가 변화되는 것이 있다.

2.2.2 ML + IL

강화학습과 모방학습을 합쳐 인간 흉내를 내는 강화학습 에이전트를 만들어낸다. 단, 에이전트가 인간처럼 행동할 뿐이지 임무수행능력을 조절하진 못 한다.

※ 본인이 진행하는 연구와 유사하거나 관련이 있는 선행 연구에 대한 조사 결과를 기술한다.
※ 참고 자료에 제시된 본문을 그대로 활용하면 안 되고, 자신의 표현으로 새롭게 재구성하여 작성하여야 하며, 출처를 분명하게 밝혀야 한다
※ 본 chapter 작성을 위해 참고된 자료들은 모두 Reference 에 제시해야 한다
※ 중간보고서 대비 차이점은, 연구 결과물이 나왔으므로, 선행 연구와 본인의 연구 결과외의 차별점 등을 기술한다

3 연구 결과

3.1 문제 정의

3.1.1 강화학습

강화학습은 에이전트에게 보상을 제공하며 특정 행동을 강화하도록 유도하는 학습법이다. 이러한 기법의 성질 덕분에 에이전트는 특정 행동을 끝도 없이 강화하여 결국 사람의 수준을 넘어서는 수행 능력을 보이게 된다.

에이전트가 최고의 수행 능력을 가지게 된다는 점은 결점으로 보이는 것이 없어 보인다. 그러나 물론 장점은 곧 단점이 된다. 사람의 수준을 넘어서는 수행 능력은 사람이 에이전트와 경쟁하는 것이 성립하지 않다는 것을 의미한다. 이는 특정 분야에서 골치 아픈 문제로 작용하는데 특히 게임 분야에서 그렇다. 사람이 게임 에이전트와 경쟁하려 하는데 전혀 이길 수 없다면 재미라는 게임의 본질적인 가치가 손상되게 된다.

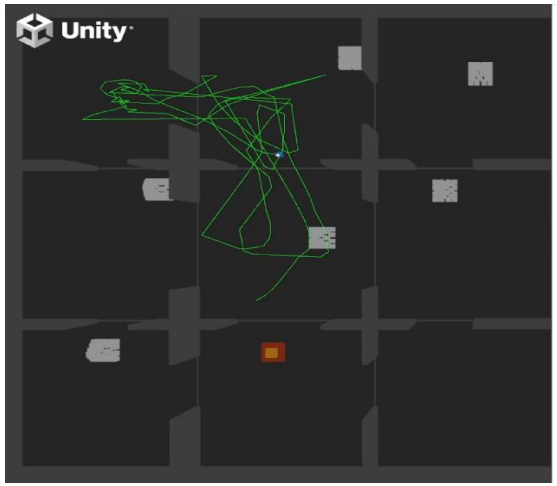
3.1.2 게임 속 강화학습

2023

Artificial Hunter Vision

[프로젝트 LINK]

개요	가상 시뮬레이션 엔진을 이용한 인공지능 학습
참여	3 Programmer
역할	Unity, Unity ML-Agent, Machine Learning, Flask
기간	2024.07~2024.10
환경	Unity, Pytorch, Flask
비고	4학년 AI 집중교육 수업
	2023-2 Ajou Softcon 연구 부문 최우수상
	회고록: https://anbak98.tistory.com/19



↓↓↓원본 텍스처 ↓↓↓

>>>

↓↓↓얘로 학습중 ↓↓↓

2024

게임 마왕 철학

개요	게임 속 마왕은 철인에 가까운가를 탐구한 문서
참여	개인
기간	2024.04
환경	Word
비고	4학년 서양고전철합 수업 문서 별첨

▪서론

나는 문득 문학 작품 속 마왕이 플라톤이 정의한 철인에 가깝다는 생각이 들었다. 본 에세이에서는 이러한 생각을 토대로 내 생각을 전개할 것이다. 다만 그 전에 마왕의 개념을 정립할 필요가 있다.

마왕이란 개념은 불교에서 유래된 호칭이다. 마왕이란 중생이 불도에 들어가는 것을 방해하는 귀신으로 석가모니가 수행할 때 그를 방해한 마리 파파야스가 그러하다. 불교는 선 악의 이분법을 부정하기 때문에 마왕이 악한 존재라고 할 순 없으며 그저 번뇌에 사로잡혀버린 존재 혹은 번뇌 그 자체로 생각할 수 있다. 현재로 넘어와 마왕이란 개념은 많은 문학 작품의 단골 소재로 쓰인다. 나를 비롯한 지금 세대 청년들에겐 본래의 종교적 의미보다는 게임 및 소설 속 마왕에 더 익숙하다. 문학 속 마왕은 정말 다양한 모습을 가지고 있다. 현대 문학 속 마왕은 절대악, 필요악, 순수악, 억울하게 누명을 쓴 사람, 선한 존재, 인간에게 우호적인 무언가 등으로 재정의 된다. 하지만 수많은 변형 속에서도 일관되게 유지되는 특징이 있다.

첫 번째로, 마왕은 자신만의 고유한 신념을 가지고 있고 이를 실제로 실천한다. 그 과정에서 무슨 일이 있더라도 그 신념이 꺾이는 일은 없다. 두 번째로, 마왕을 절대적으로 신뢰하고 충성하는 우수한 부하들이 많다. 그들은 죽음조차 충성으로 받아들이는 추종자들이며, 마왕에 대한 충성을 제외하곤 어떠한 약점 잡힐 요소(가족, 연인 등)도 가지고 있지 않다. 세 번째로, 평범한 계급의 마왕 세력 시민들은 마왕과 마왕군을 추종하고 마왕 세력의 기반을 다지는데 충실하다.

물론, 이를 벗어나는 작품들도 무척 많다. 때문에 현대에서 마왕이란 개념을 통틀어 분석하기란 쉽지 않다. 마왕의 근간이 되는 불교의 마왕을 공부하기에는 에세이의 성격과 맞지 않다. 엉뚱한 얘기지만 만약 번뇌 그 자체가 마왕이라면 플라톤의 철인과 얼마나 동떨어졌고, 어떻게 다를지 해석하는 것도 재밌을 것 같다. 그러나 많은 마왕들을 자세히 들여다보면 글이 지저분하질 수 있기에, 먼저 내 주관에 따라 정의한 마왕의 세 가지 공통적 특징을 살펴보고자 한다.

▪다른 종류의 이데아, 마왕

마왕들은 자신만의 꺾이지 않는 고유한 신념을 가지고 있다. 내가 어렸을 적 본 마왕은 악의 상징으로 세계정복을 꿈꾸고 끝없이 전쟁을 벌이는 존재였다. 그 어떤 참혹한 현장, 간계, 외교도 마왕의 의지를 꺾지 못한다. 마왕의 의지가 너무나 강력한 나머지, 그 세계 속 사람들은 모든 외교적 시도를 포기하고 오직 죽임으로써 마왕의 의지를 꺾으려 한다.

이 세계 속 마왕은 보통 '호모 사피엔스'나 '포유강 영장목 사람상과 사람과 사람속에 속하는 동물'같은 걸로 분류되지 않고 '마물' 혹은 '악마'와 같은 생물로 분류된다. 때문에 인간 vs 마왕의 종족 차원의 대립 구도가 이루어져 있으며 두 종족 사이에는 맹목적인 적대감과 공격성만이 있을 뿐이다. 이 두 세력이 어쩌다 싸우게 되었는가 같은 질문에는 보통 '인간이 살던 땅에 마왕이 나타났다'라고 답한다. 마왕의 유래에 대해서는 자세히 다루지 않는 편이다.

작품은 대부분이 철저한 인간 시점으로 진행된다. 마왕의 무차별 학살에 대항해 인간 용사가 마왕을

2024

게임에 진리란 있는가?

개요	게임에 진리란 있는가에 대한 의문에서 시작되는 철학
참여	개인
기간	2024.06
환경	Word
비고	4학년 서양고전철합 수업 기말
	문서 별첨
	블로그 링크

Kaggle

- 개요: 오로지 100K Parameters만으로 최적의 Computer Vision Model 구축하기
- 인원: 개인
- 기간: 2023.09.13-2023.10.17
- 비고: Score 0.626 (Professor says Good score)
- 개발 환경: Pytorch

아주대학교 수강목록

2019

수학1 A

글쓰기 C+

아주인성 C+

SW커리어세미나 P

영어2 B

컴퓨터프로그래밍실습 A

영어1 B

이산수학 B

현대사회의윤리 C+

창의소프트웨어입문 B

물리학실험 C+

물리학 C+

2020

컴퓨터구조 A

디자인과 사회 B+

객체지향프로그래밍및실습 B

시리어스게임분석 B

데이터베이스 B

선형대수1 B+

알고리즘 B

자료구조및실습 B+

미디어와창업 B+

2021

컴퓨터그래픽스 B+

확률 및 통계1 C

기계학습 B

컴퓨터네트워크 B

운영체제 C+

미래산업혁명과 기술창업론 B+

디지털회로 C+

인공지능 B

데이터마이닝 B

아주도전1 B+

아주도전7 B

아주도전9 B

생명과학 C+

자기주도프로젝트 A

2023

오픈소스SW입문 B

인디게임제작 A+

인간과컴퓨터상호작용 C+

AI집중교육1 A+

AI집중교육2 B

2024

드로잉연습 B

서양고전철학 B+

미디어소프트웨어엔지니어링 B+

시스템설계및실습 A

시리어스게임프로젝트 A+

파란학기 A